

Claude Code Deep Dive Workshop

Chapter 3 - Admin Setup

Enterprise 배포, 자격증명, 거버넌스 완전 가이드

Choi, WooHyung

Prin. Solutions Architect

AWS Korea

whchoi@amazon.com



Agenda

Claude Code Workshop

01

배포 전략

npm 미러, Native, Docker, MSI 선택

02

자격증명 관리

API Key, Bedrock, Vertex AI, OAuth

03

네트워크와 보안

프록시, 인증서, 화이트리스트

04

거버넌스와 정책

managed-settings, 권한 강제, 정책 계층

05

모니터링과 비용 통제

OTel, /cost, 예산 알람

06

SSO와 신원 통합

IAM Identity Center, OIDC, SAML

07

감사 / 컴플라이언스 / Lab

SIEM, 트러블슈팅, 3 Labs

CHAPTER 03

Admin Setup

Enterprise 배포와 거버넌스



Learning Objectives

Admin이 챕터를 마치고 할 수 있는 것

OBJ 1

배포 전략 수립

npm 미러, Native 패키지, Docker, MSI 중 조직에 맞는 방식을 선택합니다.

OBJ 2

자격증명 통합

AWS Bedrock 또는 Vertex AI 경로로 사내 IAM과 통합합니다.

OBJ 3

거버넌스 구축

조직 정책, 권한 규칙, 사용량 한도를 settings로 강제합니다.

OBJ 4

감사와 모니터링

사용 로그를 사내 SIEM으로 보내고 비용을 실시간 추적합니다.

OUTCOME

챕터를 마치면 본인 조직의 보안 정책과 컴플라이언스 요건을 만족하는 Claude Code 배포 계획을 수립하고 실행할 수 있게 됩니다.

4

학습 목표

9 Parts

구성

3 Labs

실습 랩

Enterprise

대상 수준

Admin이 마주하는 도전

도입 시 흔히 발생하는 4가지 문제

PAIN 1

자격증명 누출

개발자 PC에 API Key가 평문으로 저장되어 보안 사고 위험이 큼.

PAIN 2

비용 폭주

통제 없이 도입하면 Opus 남발로 월 비용이 예산의 5배 이상 나옴.

PAIN 3

권한 분산

개발자마다 다른 settings로 같은 도구도 다르게 동작.

PAIN 4

감사 추적 불가

누가 무엇을 언제 했는지 기록이 없어 컴플라이언스 감사를 통과할 수 없음.

THE GOAL

Chapter 3은 이 4가지 도전 모두에 답합니다. 중앙 집중식 자격증명, 예산 통제, 권한 강제, 감사 로그를 한 번에 구축합니다.

Centralize

중앙 집중

Control

통제

Enforce

강제

Audit

감사

CHAPTER 03 - PART 01

Admin Setup - 배포전략

Enterprise 배포와 거버넌스



Part 1 / 배포 전략

조직에 맞는 설치 방식 선택

PART OBJECTIVE

npm, Native, Docker, MSI 4가지 배포 방식의 장단점을 비교하고, 사내 미러 운영, 자동 업데이트 통제, 버전 고정 같은 운영 패턴까지 학습.

TOPIC 1

4가지 배포 방식

npm, Native, Docker,
MSI 각 방식의 장단점과 적
합한 조직 환경을 다룹니다.

TOPIC 2

사내 미러 운영

Verdaccio, Nexus, JFrog
Artifactory로 내부 패키지
미러를 구축합니다.

TOPIC 3

버전 통제

특정 버전 고정, 자동 업데이
트 차단, 점진 롤아웃 전략을
다룹니다.

TOPIC 4

대량 배포

MDM, GPO, Ansible 같은
도구로 수백 대 PC에 일괄
배포합니다.

15 Slides

Part 1 분량

~30 min

예상 강의 시간

4 Methods

배포 방식

Production

실무 적용 수준

4가지 배포 방식

조직 환경별 적합도

Method	장점	적합한 조직
npm 글로벌	간편, 업그레이드 쉬움, Node 친화적	개발자 위주, Node 환경 표준
Native Installer	Node 불필요, 일반 사용자도 가능	비개발자 포함, 표준 사용자
Docker	완전 격리, CI/CD 친화, 재현성 100%	컨테이너 친화, 자동화 위주
Windows MSI	GPO 배포, 사일런트 설치 가능	Windows 표준 조직

RECOMMENDATION

대부분의 엔터프라이즈는 Native + MSI 조합이 표준. 개발자 일부에게만 npm을 추가 제공하는 패턴이 흔함.

npm

Dev-friendly

Native

Standard

Docker

CI/CD

MSI

Windows GPO

사내 npm 미러 운영

외부 의존성 차단과 통제

Verdaccio

```
# 1. Verdaccio 사내 미러 구축 (간단한 옵션)
$ npm install -g verdaccio
$ verdaccio --listen 4873

# 2. config.yaml에서 anthropic 패키지 캐싱
storage: ./storage
packages:
  '@anthropic-ai/*':
    access: $authenticated
    publish: $authenticated
    proxy: npmjs # 첫 요청 시 외부에서 캐싱
  '**':
    access: $authenticated
    publish: $authenticated
    proxy: npmjs

# 3. 사내 PC에 npm 레지스트리 설정 (GPO/Ansible로 일괄)
$ npm config set registry https://npm.internal.company.com

# 4. 설치 시 사내 미러에서 가져옴
$ npm install -g @anthropic-ai/claude-code

# 5. 보안 효과
- 외부 npm 직접 접근 불필요
- 사내 미러에서 보안 스캔 후 제공
- 버전 회수(yank) 시 즉시 차단 가능
```

Verdaccio

추천 미러

GPO/Ansible

레지스트리 설정 배포

Scan

보안 스캔 적용

Cached

오프라인 가능

Nexus / JFrog Artifactory

엔터프라이즈 표준 레지스트리

NEXUS

Sonatype Nexus Repository

1. 오픈소스 OSS 버전 무료
2. npm, maven, pypi, docker 통합
3. proxy + hosted + group 구조
4. 보안 스캔: IQ Server 연동
5. 운영: 사내 표준 패키지 관리

JFROG

JFrog Artifactory

1. 상용, Enterprise 표준
2. 30+ 패키지 형식 지원
3. Xray 보안 스캔 통합
4. 고가용성, 멀티 리전 복제
5. 대규모 조직에 적합

Nexus OSS

무료 옵션

Artifactory

엔터프라이즈

Scan

취약점 스캔 통합

Mirror

npm 미러 표준

Native Installer 대량 배포

macOS와 Linux 환경

Mass deploy

```
# macOS - MDM (Jamf Pro 등)
1. claude-code.pkg 다운로드
2. Jamf Pro에 패키지 업로드
3. 정책 생성: 그룹 대상, 사일런트 설치
4. self-service에도 게시

# Linux - Ansible playbook
# install_claude.yml
- hosts: developers
  become: yes
  tasks:
    - name: Download installer
      get_url:
        url: https://claude.ai/install.sh
        dest: /tmp/claude_install.sh
        mode: '0755'
    - name: Run installer
      shell: /tmp/claude_install.sh
      environment:
        CLAUDE_INSTALL_PATH: /opt/claude
    - name: Verify
      command: claude --version

# Ansible 실행
$ ansible-playbook -i hosts install_claude.yml
```

Jamf
macOS MDM

Ansible
Linux 배포

Silent
무인 설치

Verify
설치 검증

Windows MSI / GPO 배포

도메인 환경 일괄 설치

GPO / Intune

```
# 1. MSI 다운로드 및 검증
$ Invoke-WebRequest \
  -Uri https://claude.ai/download/windows.msi \
  -OutFile \\fileservice\Software\ClaudeCode\claude.msi
$ Get-FileHash claude.msi # SHA256 확인

# 2. GPO 생성 (Group Policy Management Console)
# Computer Configuration → Policies → Software Settings
#   → Software Installation → New → Package
#   UNC path: \\fileservice\Software\ClaudeCode\claude.msi
#   Deployment: Assigned

# 3. 또는 Intune (modern, 클라우드 기반)
# Apps → Add → Windows app (Win32)
# .intunewin 패키지 생성 후 업로드
# Assignment: Required for 그룹

# 4. 사용자 PC 부팅 시 자동 설치
# 사용자 작업 없이 백그라운드 진행

# 5. 사일런트 옵션 (수동 설치 시)
$ msixexec /i claude.msi /quiet /norestart
```

GPO

AD 도메인

Intune

클라우드

/quiet

사일런트

Hash

무결성 검증

Docker 엔터프라이즈 이미지

사내 표준 이미지 구축

```
# Dockerfile - 사내 표준 이미지
FROM anthropic/claude-code:1.5.0 # 버전 고정

# 사내 CA 인증서 설치
COPY company-ca.crt /usr/local/share/ca-certificates/
RUN update-ca-certificates

# 사내 자주 사용하는 CLI 추가
RUN apt-get update && apt-get install -y \
    jq ripgrep awscli git-lfs gh \
    && rm -rf /var/lib/apt/lists/*
# 사내 표준 CLAUDE.md 템플릿
COPY company-CLAUDE.md /etc/claude/templates/CLAUDE.md

# 사내 MCP 서버 사전 설정
COPY mcp-config.json /etc/claude/mcp.json
```

Enterprise Dockerfile

```
# 환경변수 사전 설정
ENV CLAUDE_CONFIG_DIR=/etc/claude
ENV NODE_EXTRA_CA_CERTS=/usr/local/share/ca-certificates/company-ca.crt
ENV HTTPS_PROXY=https://proxy.company.com:8080

WORKDIR /workspace
ENTRYPOINT ["claude"]
```

Pin version

버전 고정

CA cert

사내 인증서

CLAUDE.md

표준 템플릿

Proxy

프록시 사전 설정

버전 고정과 통제

자동 업데이트 차단

Version Control

1. npm 설치 시 특정 버전 고정
\$ npm install -g @anthropic-ai/claude-code@1.5.0

2. 자동 업데이트 체크 비활성화 (모든 사용자)
사내 표준 환경변수로 강제
\$ export CLAUDE_DISABLE_UPDATE_CHECK=1

3. managed-settings.json (관리자만 수정 가능)
{
 "updateChecker": {
 "enabled": false,
 "notifyOnUpdate": false
 },
 "fixedVersion": "1.5.0"
}

4. 새 버전 검증 후 일괄 롤아웃 (Ansible 예시)

```
- name: Update Claude Code to 1.6.0  
  hosts: developers  
  tasks:  
    - name: Install new version  
      npm:  
        name: '@anthropic-ai/claude-code'  
        version: '1.6.0'  
        global: yes
```

5. 검증 절차: dev → staging → production 그룹별 단계 배포

@1.5.0

명시 버전

DISABLE_UPDATE

체크 차단

managed-settings

관리자 통제

Staged

단계 배포

점진 롤아웃 전략

안전한 버전 업그레이드

STEP 01

Lab 환경
검증 (1-2일)

STEP 02

Early adopter
그룹 배포

STEP 03

50% 사용자
배포 + 모니터

STEP 04

전체 배포

STEP 05

롤백 시
자동 회복

Stage	대상	기간
Lab	내부 검증 PC 5-10대	1-2일
Canary	Early adopter 10-20명	3-5일
Partial	50% 사용자	1주
Full	전체 사용자	확정

4 stages

롤아웃 단계

Canary

조기 위험 감지

Monitor

단계마다 모니터

Rollback

즉시 복귀 가능

오프라인/에어갭 환경

인터넷 없는 환경 배포

Air-gapped

시나리오: 보안 정책상 인터넷 차단된 사내망

1. 외부 망에서 패키지 수집 (이동 가능 미디어)

\$ npm pack @anthropic-ai/claude-code

claude-code-1.5.0.tgz 생성

\$ docker save anthropic/claude-code:1.5.0 -o claude.tar

tar 파일로 패키지화

2. 사내 미러 서버로 이전 (오프라인 매체)

USB 또는 보안 데이터 이전 채널 사용

3. 사내 미러에 등록

Verdaccio

\$ npm publish claude-code-1.5.0.tgz --registry https://internal-npm

Docker

\$ docker load -i claude.tar

\$ docker tag anthropic/claude-code:1.5.0 registry.internal/claude:1.5.0

\$ docker push registry.internal/claude:1.5.0

4. 사내 사용자는 사내 미러에서만 설치

외부 접근 없이 완전한 오프라인 동작

5. 단, Anthropic API 호출은 별도 게이트웨이 필요

Bedrock VPC Endpoint 또는 PrivateLink 활용

npm pack

npm tarball

docker save

Docker tar

Internal mirror

오프라인 미러

PrivateLink

API 게이트웨이

멀티 플랫폼 전략

혼합 OS 환경 대응

WINDOWS

Windows 데스크톱

MSI + GPO 또는 Intune.
WSL2 사용자에게는 npm
추가 옵션.

MAC

macOS 노트북

pkg + Jamf MDM.
Homebrew 사용자에게는
brew 옵션 추가.

LINUX

Linux 서버

install.sh + Ansible.
Docker로 격리 옵션도 함께
제공.

CI/CD

CI/CD 환경

Docker 이미지로 통일.
GitHub Actions, GitLab
CI 표준 이미지화.

KEY

플랫폼별 도구는 다르지만 결과적으로 동일한 Claude Code와 동일한 settings.json을 사용.
사용자 경험을 OS와 무관하게 통일하는 것이 목표.

MSI/GPO

Win

pkg/Jamf

Mac

sh/Ansible

Linux

Docker

CI

라이선스와 약관

엔터프라이즈 계약 고려사항

항목	내용	확인 필요
Claude Code 라이선스	Anthropic 약관 + 사용량 기반 청구	법무팀 검토
API 사용량	입력/출력 토큰 단위, 모델별 단가 상이	재무팀 예산
데이터 처리	기본은 ZDR 옵션, 학습 미사용	DPA 체결 필요
Cloud 경로	Bedrock/Vertex는 해당 클라우드 청구	기존 계약 확장
Open source 의존성	MIT, Apache 등 호환 라이선스	OSS 정책 검토

Anthropic

직접 계약

DPA

데이터 처리 합의

Bedrock

AWS 계약 확장

Vertex

GCP 계약 확장

사용량 예측

도입 전 비용 추정

Forecast

1. 개발자 1명당 월 평균 (실측 기준)
- 평일 사용: 6시간 × 20일 = 120시간/월
- 시간당 평균 토큰: input 50K, output 5K
- 월 토큰: input 6M, output 0.6M
- Sonnet 단가: \$3/M input + \$15/M output
- 1인당 월 비용: $6 \times \$3 + 0.6 \times \$15 = \$27$

2. 조직 규모별 예상 월 비용
- 10명 팀: \$270/월
- 50명 팀: \$1,350/월
- 200명 조직: \$5,400/월

3. 비용 증감 요인
+ Subagent 다용: +30-50% (병렬 호출)
+ Opus 사용: +200-400% (해당 작업)
- Haiku 적극 활용: -40-60%
- Plan 모드 사용: -10-20% (의도 정확화)

4. 안전 마진 권장
실제 도입 시 예상치 + 30% 버퍼 확보

\$27/dev

월 평균 1인

+30%

안전 마진

Variable

실제 변동 큼

Monitor

도입 후 추적

배포 전 체크리스트

실수 없이 진행하기 위한 8가지 확인 사항

No	Item	담당
1	배포 방식 결정 (npm/Native/Docker/MSI)	IT/DevOps
2	사내 미러 또는 레지스트리 구축	IT/DevOps
3	계약 및 라이선스 검토 (DPA 포함)	법무
4	비용 예산 확보 (1인당 \$27 + 30% 버퍼)	재무
5	자격증명 방식 결정 (Key/Bedrock/Vertex)	IT 보안
6	권한 정책 초안 (managed-settings.json)	IT 보안
7	롤아웃 단계 계획 (Lab → Canary → Full)	PM
8	교육 자료 준비 (워크샵 또는 단축 가이드)	L&D/SA

8 items
체크리스트

5 owners
담당 부서

Pre-launch
런칭 전

Sign-off
모두 완료 후 진행

Part 1 Recap

배포 전략 핵심 정리

TAKEAWAY 1

4가지 방식

npm/Native/Docker/MS
I 중 조직 환경에 맞게 선택 (혼합 가능)

TAKEAWAY 2

사내 미러 운영

Verdaccio/Nexus/Artifactory로 외부 의존 차단 및 보안 강화

TAKEAWAY 3

버전 통제

managed-settings로 자동 업데이트 차단, 단계별 롤아웃

TAKEAWAY 4

비용 예측

1인당 월 \$27 기준 + 30%
안전 마진, 변동 요인 고려

NEXT - PART 2

Part 2는 자격증명 관리입니다. API Key 분배 위험, AWS Bedrock 통합, Vertex AI 경로, OAuth SSO를 15장에 걸쳐 다룹니다.

Part 1

완료

Part 2

Credentials (15p)

Part 3

Network (15p)

~3.5h

잔여 시간

CHAPTER 03 - PART 02

Admin Setup – 자격증명 관리

Enterprise 배포와 거버넌스



Part 2 / 자격증명 관리

API Key부터 SSO까지

PART OBJECTIVE

Claude Code의 4가지 자격증명 방식을 비교하고, 엔터프라이즈 환경에 가장 적합한 패턴을 학습합니다. AWS Bedrock과 Vertex AI 통합도 다룹니다.

METHOD 1

Anthropic API Key

직접 키 분배. 가장 간단하지만 누출 위험이 가장 큼.

METHOD 2

AWS Bedrock

IAM Role 기반, 사내 AWS 계정과 통합, Enterprise 표준.

METHOD 3

Google Vertex AI

GCP ADC, IAM 통합, GCP 사용 조직에 적합.

METHOD 4

OAuth (Pro/Max)

개인 Pro 구독, SSO 통합 가능, 소규모 조직 적합.

15 Slides

Part 2 분량

~30 min

예상 강의 시간

4 Methods

자격증명 옵션

Security

핵심 주제

API Key 분배의 위험

왜 엔터프라이즈에서 권장하지 않는가

RISK 1

평문 저장

환경변수나 dotfile에 평문으로 저장되면 노트북 분실 시 즉각 누출.

RISK 2

회수 어려움

직원 퇴사 시 키를 일일이 회수하기 어려워 unused active 키가 누적.

RISK 3

Git 커밋 사고

.env 파일이 실수로 Git에 커밋되어 공개 저장소에 노출되는 사고가 빈번.

RISK 4

감사 추적 한계

키 자체만으로는 누가 사용했는지 추적이 제한적이라 감사 요건 충족이 어려움.

RECOMMENDATION

10명 이상 조직은 반드시 Bedrock 또는 Vertex 경로를 사용. API Key 분배는 1-5명 PoC 단계에서만 일시적으로 허용.

Plaintext

저장 방식 위험

Revocation

회수 어려움

Git leak

커밋 사고

Audit gap

감사 한계

AWS Bedrock 설정 개요

Enterprise 표준 패턴

STEP 01

Bedrock
모델 액세스

STEP 02

IAM Role
또는 사용자

STEP 03

환경변수
사내 표준화

STEP 04

Claude Code
자동 라우팅

STEP 05

비용
AWS 청구 통합

WHY BEDROCK

AWS Bedrock 경로는 IAM 기반 자격증명, 기존 보안 정책 활용, AWS 청구 통합, VPC Endpoint 지원, CloudTrail 감사 등 엔터프라이즈에 필요한 모든 요소를 제공.

IAM

신원 통합

AWS 청구

비용 통합

VPC EP

사설망

CloudTrail

감사 로그

Bedrock 모델 액세스 신청

시작 전 필수 절차

Model Access

```
# 1. AWS Console → Bedrock → Model access
# Anthropic 모델 패밀리 액세스 요청
# - Claude Sonnet 4.6
# - Claude Opus 4.7
# - Claude Haiku 4.5

# 신청 시 입력 항목
- 회사명 / 사용 사례 / 데이터 처리 방식
- 보통 1시간 이내 자동 승인 (일부 모델 별도)

# 2. 리전 선택 (한국 권장)
- ap-northeast-2 (Seoul) - 모델 가용성 확인
- us-east-1 (Virginia) - 가장 빠른 모델 출시
- us-west-2 (Oregon) - 두 번째 빠른 출시

# 3. 액세스 후 확인
$ aws bedrock list-foundation-models \
  --region ap-northeast-2 \
  --by-provider anthropic

# 4. IAM 정책 준비 (다음 슬라이드)
```

Console

신청 위치

~1h

승인 시간

Region

가용성 다름

Verify

CLI 확인

IAM 정책 설계

Bedrock 접근 최소 권한

```
# bedrock-claude-code-policy.json
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "InvokeAnthropicModels",
      "Effect": "Allow",
      "Action": [
        "bedrock:InvokeModel",
        "bedrock:InvokeModelWithResponseStream"
      ],
      "Resource": [
        "arn:aws:bedrock:::foundation-model/anthropic.claude-sonnet-4-5*",
        "arn:aws:bedrock:::foundation-model/anthropic.claude-haiku-4-5*"
      ]
    },
    {
      "Sid": "ListModel",
      "Effect": "Allow",
      "Action": "bedrock:ListFoundationModels",
      "Resource": "*"
    }
  ]
}
```

적용 방법

- 사용자 IAM Group에 attach (가장 일반적)
- 또는 IAM Role + AssumeRole (단기 자격증명)
- Opus는 별도 정책으로 분리 (비용 통제)

IAM Policy

InvokeModel

필수 액션

Resource ARN

모델 단위 제한

Group

일반 분배

Role

단기 자격증명

Bedrock 환경 설정

Claude Code에 알리는 방법

1. 환경변수 설정 (개발자 PC 표준화)

```
export CLAUDE_CODE_USE_BEDROCK=1
export AWS_REGION=ap-northeast-2
export AWS_PROFILE=default
```

2. 또는 ~/.aws/config 명시 (AWS CLI 표준)

```
[profile claude-code]
region = ap-northeast-2
role_arn = arn:aws:iam::123:role/ClaudeCodeRole
source_profile = sso-user
```

3. 자격증명은 다음 순서로 자동 감지

1. 환경변수 (AWS_ACCESS_KEY_ID 등)
2. ~/.aws/credentials 프로파일
3. EC2 Instance Profile (서버 환경)
4. ECS Task Role (컨테이너)
5. EKS IRSA (쿠버네티스)

4. 동작 검증

```
$ claude doctor
Auth:
  Provider: AWS Bedrock           ✓
  Region:   ap-northeast-2        ✓
  Profile:   claude-code           ✓
  Model access: Sonnet, Haiku (Opus denied)  i
```

Env config

USE_BEDROCK=1

경로 활성화

AWS_REGION

리전 명시

AWS_PROFILE

자격증명

doctor

검증

Bedrock + SSO 통합

IAM Identity Center 연동

IAM IC + SSO

1. AWS Organizations + IAM Identity Center 활성화
(구 AWS SSO)

2. Permission Set 생성: ClaudeCodeUser
- 정책: bedrock-claude-code-policy 첨부
- 세션 시간: 8시간 (업무 시간)

3. 사용자 또는 그룹에 할당
- 개발팀 그룹 → ClaudeCodeUser 권한
- 시니어 그룹 → ClaudeCodeAdvanced (Opus 포함)

4. 개발자 PC에서 SSO 로그인
\$ aws sso login --profile claude-code
브라우저 OAuth 로그인 → 임시 자격증명 발급

5. Claude Code 사용
\$ claude
자동으로 SSO 자격증명 사용
만료 시 자동 갱신 또는 재로그인 안내

6. 보안 이점
- 영구 API Key 불필요
- 8시간 후 자동 만료
- 사용자 신원 추적 가능 (CloudTrail)
- 퇴사 시 권한 즉시 회수

IAM IC
Identity Center

Permission Set
권한 묶음

8h
세션 시간

No long-lived key
영구 키 없음

OAuth (Pro/Max) 옵션

소규모 조직 또는 개인 사용

OAuth flow

```
# 1. Anthropic 계정 생성 + 결제 카드 등록
# https://claude.ai 에서 Pro 또는 Max 구독

# 2. Claude Code에서 OAuth 로그인
$ claude
> /login
# 브라우저가 열리며 OAuth 인증 진행
# 성공 시 토큰이 ~/.claude/oauth.json에 저장

# 3. 가입 플랜에 따른 차이
- Pro: 표준 사용량 (개인 개발 적합)
- Max: 더 큰 컨텍스트 사용량, 우선 처리
- Team/Enterprise: 별도 협의

# 4. 토큰 자동 회전 (시간제 만료 후)
- access_token: 1시간
- refresh_token: 자동 갱신
```

```
# 5. 적합한 경우
- 1-5인 스타트업, 작은 팀
- PoC, 트라이얼 단계
- 개인 개발자

# 6. 제한사항
- 사내 SSO 통합 불가
- IAM 기반 권한 통제 어려움
- 감사 로그가 Anthropic 측에만 남음
```

/login

로그인 명령

Pro/Max

구독 플랜

Auto-refresh

토큰 자동 갱신

Small org

소규모 적합

자격증명 의사결정

조직 상황별 최적 선택

STEP 01

AWS 사용
증인가?

STEP 02

GCP 사용
증인가?

STEP 03

10명 이상
조직?

STEP 04

PoC
단계인가?

STEP 05

결정

조건	권장	비고	
AWS 사용 + 엔터프라이즈	AWS Bedrock + IAM Identity Center	최고 보안	
GCP 사용 + 엔터프라이즈	Vertex AI + GCP IAM	GCP 통합	
10인 이하 + Cloud 미사용	OAuth Pro/Max (개인 계정)	간단한 시작	
PoC, 트라이얼	API Key (1-2주 한정)	이후 전환 계획	
Bedrock AWS 표준	Vertex GCP 표준	OAuth 소규모	API Key PoC only

PrivateLink & VPC Endpoint

사내망 트래픽 격리

VPC Endpoint

시나리오: 모든 트래픽이 인터넷 거치지 않고 AWS 사설망 내에서

1. VPC Endpoint 생성 (Bedrock용)
AWS Console 또는 CloudFormation

Resources:

BedrockVPCEndpoint:

Type: AWS::EC2::VPCEndpoint

Properties:

VpcId: !Ref MyVPC

ServiceName: com.amazonaws.ap-northeast-2.bedrock-runtime

VpcEndpointType: Interface

SubnetIds:

- !Ref PrivateSubnet1
- !Ref PrivateSubnet2

SecurityGroupIds:

- !Ref BedrockEndpointSG

PrivateDnsEnabled: true

2. DNS Resolution

bedrock-runtime.ap-northeast-2.amazonaws.com

→ VPC 내부 IP로 자동 해석 (인터넷 우회)

3. 효과

- API 트래픽이 인터넷 거치지 않음
- 사내 네트워크 내에서만 통신
- 사내 보안 정책 컴플라이언스 만족
- VPC Flow Logs로 트래픽 감사

Interface

엔드포인트 유형

Private DNS

자동 해석

Internal-only

인터넷 우회

Flow Logs

감사 가능

자격증명 회전

주기적 갱신 자동화

API KEY

Anthropic Key

수동 회전, 분기 1회 또는 누출 시 즉시. 회수 후 새 키 분배 필요.

IAM USER

AWS Access Key

90일마다 자동 알람, 회전 스크립트로 자동화 가능.

SSO

IAM Identity Center

세션 만료 8시간, 회전 불필요. 자동 토큰 갱신.

OAuth

Pro/Max OAuth

access token 1시간, refresh token 자동 갱신. 사용자 액션 없음.

RECOMMENDATION

장기 자격증명(API Key, IAM Access Key)을 피하고 단기 자격증명(SSO, OAuth) 사용을 우선.

회전이 자동인 옵션이 운영 부담이 적음.

Quarterly

API Key

90 days

IAM Key

8h auto

SSO

1h auto

OAuth

Secret 관리 도구

키 저장과 분배 자동화

AWS

Secrets Manager

AWS 환경 표준.

자동 회전, IAM 통합, 사용자 자동 알림 지원.

HASHI

HashiCorp Vault

벤더 중립, 동적 자격증명, 멀티 클라우드 지원.

GCP

Secret Manager

GCP 환경 통합. Vertex AI와 자연스러운 연동.

PATTERN

API Key를 평문으로 분배하지 말고 Secret 관리 도구에 저장 후 환경변수 주입 스크립트로 런타임에 가져오는 패턴이 표준입니다.

SM

AWS

Vault

HashiCorp

GSM

GCP

Runtime

런타임 주입

Lab 환경 자격증명

워크샵에서 적용하는 패턴

Workshop creds

시나리오: 50명 워크샵 참가자에게 임시 자격증명 분배

패턴 A: AWS IAM Identity Center 임시 권한

1. workshop-attendees 그룹에 Bedrock 정책 부여

2. 시작 시 SSO URL 공유

3. 워크샵 종료 후 그룹 제거 (권한 즉시 회수)

패턴 B: 일회용 API Key 분배 (PoC 용도)

1. 50개 키 미리 생성 (anthropic console)

2. 워크샵 시작 시 1인 1키 분배 (slack DM)

3. 워크샵 종료 후 모든 키 즉시 폐기

패턴 C: 공용 데모 계정 (가장 단순)

1. 데모용 단일 키로 모두 사용

2. 사용 한도 미리 설정 (\$50 한도)

3. 권장: 워크샵 끝에 키 즉시 폐기

추천: 보안 요건이 강하면 A, 간단함 우선이면 C

Lab 종료 후 체크리스트

- 모든 임시 권한 회수 확인

- 사용량 리포트 점검

- 데모 키 폐기

Pattern A

IAM IC

Pattern B

Disposable

Pattern C

Shared

Cleanup

종료 시 정리

Part 2 Recap

자격증명 관리 핵심 정리

TAKEAWAY 1

4가지 옵션

API Key, Bedrock,
Vertex, OAuth - 조직 환
경에 맞게 선택

TAKEAWAY 2

Cloud 경로 우선

10인 이상은 Amazon
Bedrock 권장 (IAM 통합 +
감사 + 청구 통합)

TAKEAWAY 3

SSO 통합

IAM Identity Center 또는
OAuth로 단기 자격증명 자
동 회전

TAKEAWAY 4

Secret 도구 활용

Secrets Manager, Vault
로 평문 저장 회피, 런타임
주입

NEXT - PART 3

Part 3은 네트워크와 보안입니다. 사내 프록시 통과, 인증서, 도메인 화이트리스트, 사내 PrivateLink를 15장에 걸쳐 다룹니다.

Part 2

완료

Part 3

Network (15p)

Part 4

Governance (15p)

~3h

잔여 시간

CHAPTER 03 - PART 03

Admin Setup – 네트워크와 보안

Enterprise 배포와 거버넌스



Part 3 / 네트워크와 보안

사내망 통합과 보안 강화

PART OBJECTIVE

사내 프록시 통과, CA 인증서 관리, 도메인 화이트리스트, 사내 DNS, 사설 라우팅 등 사내망 환경에서 Claude Code를 안전하게 운영하는 모든 패턴을 학습합니다.

TOPIC 1

프록시 통과

HTTP/HTTPS 프록시,
NO_PROXY 예외, 인증 프록시 처리 방법을 다룹니다.

TOPIC 2

인증서 관리

사내 CA 신뢰,
NODE_EXTRA_CA_CERTS, 자체 서명 인증서 처리를 다룹니다.

TOPIC 3

도메인 통제

필수 화이트리스트, 사내 DNS 분기, 외부 API 접근 통제 방법입니다.

TOPIC 4

사설망 통합

VPC Endpoint,
PrivateLink, VPN, ZTNA
환경 통합 패턴을 다룹니다.

15 Slides

Part 3 분량

~30 min

예상 강의 시간

4 Topics

주요 토픽

Network

실무 핵심

사내 프록시 통과

환경변수 표준 설정

Proxy env

```
# 1. 환경변수 표준 (모든 사내 PC 공통)
export HTTP_PROXY=http://proxy.company.com:8080
export HTTPS_PROXY=http://proxy.company.com:8080
export NO_PROXY=localhost,127.0.0.1,.company.com,169.254.169.254
```

```
# 2. Windows (관리자 PowerShell)
[Environment]::SetEnvironmentVariable(
    'HTTPS_PROXY', 'http://proxy.company.com:8080', 'Machine')
```

```
# 3. GPO로 일괄 배포 (Windows)
# Computer Configuration → Preferences →
# Windows Settings → Environment
```

```
# 4. Claude Code의 자동 인식
# Node.js 표준 환경변수를 그대로 사용
# 별도 Claude Code 설정 불필요
```

```
# 5. 검증
$ claude doctor
Network:
  api.anthropic.com via proxy.company.com:8080 ✓ (250 ms)
  Outbound HTTPS allowed ✓
  Proxy: http://proxy.company.com:8080 ✓
```

HTTP_PROXY

소문자 형태

HTTPS_PROXY

대문자 형태

NO_PROXY

예외 도메인

169.254.*

AWS 메타데이터

인증 프록시

Basic/NTLM 인증 통과

```
# 1. Basic 인증 (URL에 자격증명 포함)
export HTTPS_PROXY=http://user:pass@proxy.company.com:8080
# 단점: PC 환경변수에 평문 저장 (지양)

# 2. 환경변수 분리 (조금 안전)
export PROXY_USER=user
export PROXY_PASS=$(cat ~/.proxy_pass) # 파일에서 읽기
export HTTPS_PROXY=http://${PROXY_USER}:${PROXY_PASS}@proxy:8080
```

```
# 3. NTLM 인증 (Windows AD)
# cntlm 같은 로컬 프록시로 우회
# https://cntlm.sourceforge.net/
$ cat ~/.cntlm/cntlm.ini
Username user
Domain COMPANY
PassNTLMv2 ABC123...

Proxy proxy.company.com:8080
NoProxy localhost, .company.com
Listen 3128

# Claude는 cntlm으로
export HTTPS_PROXY=http://localhost:3128

# 4. 권장: Kerberos SSO 프록시
# 사용자 입력 없이 자동 인증
```

Auth proxy

Basic
URL 포함

NTLM
cntlm 우회

Kerberos
SSO 자동

Avoid plain
평문 회피

사내 CA 인증서

TLS Intercept 환경 대응

CA Certs

시나리오: 사내 프록시가 TLS를 가로채 사내 CA로 재서명

1. 사내 CA 인증서 파일 확보

보통 IT팀에서 제공 (company-ca.crt 또는 .pem)

2. Node.js 환경변수 설정

export NODE_EXTRA_CA_CERTS=/etc/ssl/company-ca.crt

또는 Windows

\$env:NODE_EXTRA_CA_CERTS = "C:\Certs\company-ca.crt"

3. 시스템 trust store에도 추가 (옵션, 권장)

macOS

\$ sudo security add-trusted-cert -d -r trustRoot \
-k /Library/Keychains/System.keychain company-ca.crt

Linux (Ubuntu/Debian)

\$ sudo cp company-ca.crt /usr/local/share/ca-certificates/

\$ sudo update-ca-certificates

Windows

certutil -addstore -f "ROOT" company-ca.crt

4. 검증

\$ claude doctor

TLS:

Custom CA: /etc/ssl/company-ca.crt ✓

Verify peer: enabled ✓

Handshake: TLS 1.3, ECDHE-ECDSA ✓

NODE_EXTRA_CA_CERTS

TS
핵심 환경변수

PEM

인증서 포맷

TLS intercept

사내 환경

No bypass

검증 비활성 X

도메인 화이트리스트

필수 외부 접근 도메인

Domain	용도	프로토콜
api.anthropic.com	Anthropic API 호출	HTTPS:443
bedrock-runtime.*.amazonaws.com	AWS Bedrock 모델 호출	HTTPS:443
*-aiplatform.googleapis.com	Google Vertex AI	HTTPS:443
registry.npmjs.org	npm 패키지 (미러 없을 때)	HTTPS:443
github.com	Git 통합, gh CLI	HTTPS:443
claude.ai	OAuth 로그인 (Pro/Max)	HTTPS:443

HTTPS only

443 포트만

No proxy

직접 접근 옵션

Egress FW

아웃바운드 방화벽

Allow list

명시 허용

사내 DNS 분기

도메인별 분리된 라우팅

Split DNS

시나리오: 외부 도메인은 인터넷, 내부 도메인은 사설망

1. /etc/hosts 또는 사내 DNS 서버 설정

```
api.anthropic.com      52.x.x.x   # 외부 직접
bedrock-runtime.ap-... 10.0.x.x   # VPC Endpoint 사설 IP
mcp.internal.company.com 10.1.x.x   # 사내 MCP 서버
github.enterprise.com   10.2.x.x   # GitHub Enterprise
```

2. Split DNS (사내 DNS 서버)

외부 조회: api.anthropic.com → 인터넷 라우팅

내부 조회: *.internal.company.com → 사내 IP

Bedrock VPC EP 활성화 시 자동 분기

3. Route 53 Resolver (AWS 환경)

Outbound endpoint으로 사내 DNS와 양방향 연동

4. 효과

- Bedrock 호출이 사설 경로로
- 사내 MCP 서버에 직접 접근
- 외부 API는 정상 라우팅
- 사용자 별도 설정 불필요

5. 디버깅

```
$ dig bedrock-runtime.ap-northeast-2.amazonaws.com
```

사내에서 10.x.x.x 응답이 나오면 정상

Split DNS

내외부 분기

Route 53

AWS Resolver

dig

디버깅

Transparent

사용자 비가시

VPN 환경 통합

재택 사용자 접근

STEP 01

VPN 클라이언트
연결

STEP 02

Split tunnel
또는 Full

STEP 03

Claude Code
실행

STEP 04

사내 DNS와
프록시 사용

STEP 05

정상 동작

SPLIT TUNNEL

분리 터널

1. 사내 도메인만 VPN으로
2. 외부 API는 인터넷 직결
3. 네트워크 부하 적음
4. 외부 API 빠름

FULL TUNNEL

전체 터널

1. 모든 트래픽이 VPN으로
2. 사내 정책 통일 적용
3. 감사 통합 용이
4. 외부 API 약간 느림

Split

성능 우선

Full

보안 우선

DNS

VPN DNS 사용

Always-on

상시 연결 권장

ZTNA 환경 통합

Zero Trust 네트워크 아키텍처

ZTNA CONCEPT

ZTNA는 사용자 신원과 디바이스 상태 기반으로 애플리케이션별 접근을 통제. 기존 VPN의 대안으로 빠르게 확산 중입니다.

CLOUDFLARE

Cloudflare Zero Trust

WARP 클라이언트 + Access.
사용자 신원 기반 정책으로 트래픽 통제.

ZSCALER

Zscaler ZIA/ZPA

Internet Access + Private Access.
글로벌 PoP, AI 트래픽 검사 옵션.

NETSKOPE

Netskope NewEdge

SASE 통합 플랫폼.
DLP 정책으로 데이터 유출 방지 통합.

ZTNA

아키텍처 패턴

Identity

신원 기반

Device

디바이스 검증

Per-app

앱별 통제

아웃바운드 트래픽 검사

AI 트래픽 모니터링

Egress Monitor

시나리오: 사내에서 AI API 트래픽을 검사하려는 요건

1. 검사 가능한 정보

- + 도메인 (api.anthropic.com 등)
- + 트래픽 양 (사용량 모니터링)
- + 시간대 패턴
- 요청 본문 (TLS 암호화로 불가)

2. 본문 검사가 필요한 경우 (드물지만)

- TLS Intercept 프록시 사용 (사내 CA로 재서명)
- 단점: 무결성, 성능, 신뢰 이슈
- 권장: 본문 검사 대신 클라이언트 측 로깅 우선

3. 클라이언트 측 로깅 패턴

OpenTelemetry exporter (Part 5에서 상세)

- # - 토큰 수, 모델, 도구 호출 로깅
- # - 본문은 hash만 저장
- # - 사내 SIEM으로 전송

4. Data Loss Prevention (DLP)

- 민감 데이터 (PII, 비밀번호) 입력 감지
- Hooks와 결합해 PreToolUse 단계에서 차단
- 사내 DLP 도구 (Symantec, Forcepoint 등) 연동

Domain

검사 가능

Content

TLS 암호화

OTel

클라이언트 로깅

DLP

민감 데이터 차단

Hooks를 통한 DLP

민감 정보 사전 차단

DLP Hook

```
# .claude/settings.json
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash|Edit|Write",
        "hooks": [{
          "type": "command",
          "command": "/usr/local/bin/dlp-check.sh"
        }]
      }
    ]
  }
}

# /usr/local/bin/dlp-check.sh
#!/bin/bash
INPUT=$(cat) # stdin으로 도구 호출 정보 수신
```

```
# 민감 패턴 감지
if echo "$INPUT" | grep -qE 'AKIA[0-9A-Z]{16}'; then
  echo '{"action":"block","reason":"AWS key detected"}' >&2
  exit 1
fi

if echo "$INPUT" | grep -qE 'ssh-rsa AAAAB3'; then
  echo '{"action":"block","reason":"SSH private key"}' >&2
  exit 1
fi

# 통과
exit 0

# 효과
- AWS Key 노출 즉시 차단
- SSH 개인키 노출 차단
- 모든 도구 호출 사전 검증
```

PreToolUse

Hook 시점

Block

차단 액션

Pattern

정규식 매칭

Mandatory

강제 적용

사내 MCP 서버 통합

사내 도구를 Claude Code에 연결

Internal MCP

```
# 시나리오: 사내 JIRA, Confluence, 사내 GitLab을 Claude Code가 사용
# .claude/settings.json (팀 공유)
{
  "mcpServers": {
    "jira-internal": {
      "command": "node",
      "args": ["/opt/mcp/jira-server.js"],
      "env": {
        "JIRA_URL": "https://jira.company.com",
        "JIRA_TOKEN": "${JIRA_TOKEN}"
      }
    },
    "confluence-internal": {
      "type": "http",
      "url": "https://mcp-confluence.company.com",
      "auth": {
        "type": "oauth",
        "client_id": "claude-code"
      }
    }
  },

```

```
"gitlab-internal": {
  "type": "sse",
  "url": "https://mcp-gitlab.company.com/sse",
  "headers": {
    "X-Internal-Auth": "${GITLAB_TOKEN}"
  }
}
}
```

사용 예
> 이번 스프린트의 JIRA 티켓 5개를 GitLab MR로 이동해 줘

stdio

로컬 명령

http

HTTP 서버

sse

Server-Sent

OAuth

SSO 통합

네트워크 트러블슈팅

연결 실패 시 진단 순서

STEP 01

doctor 실행

STEP 02

도메인 ping

STEP 03

프록시 통과

STEP 04

인증서 검증

STEP 05

방화벽 점검

Diagnostics

```
# 1. claude doctor로 1차 진단
$ claude doctor
# Network 섹션의 OK/FAIL 항목 확인
# 2. 기본 연결 테스트
$ curl -v https://api.anthropic.com/v1/health
# TLS handshake와 응답 확인
# 3. 프록시 통과 테스트
$ curl -v --proxy $HTTPS_PROXY https://api.anthropic.com
# 4. 인증서 검증
$ openssl s_client -connect api.anthropic.com:443 \
  -CAfile $NODE_EXTRA_CA_CERTS
# Verify return code: 0 (ok) 확인
```

```
# 5. DNS 확인
$ dig api.anthropic.com
$ dig bedrock-runtime.ap-northeast-2.amazonaws.com

# 6. 방화벽 (개발자 PC에서)
# Windows: New-NetFirewallRule (관리자)
# macOS: pf rule 확인
# Linux: iptables -L
```

doctor

1차 진단

curl

기본 테스트

openssl

인증서 검증

dig

DNS 확인

사내 보안 정책 매핑

Claude Code와 기존 정책 연결

사내 정책	Claude Code 대응	구현
API Key 평문 저장 금지	Amazon Bedrock 경로 + IAM	Cloud 자격증명
외부 도메인 화이트리스트	방화벽 + 6개 필수 도메인 허용	Egress FW
TLS 검증 필수	NODE_EXTRA_CA_CERTS 명시	인증서 설정
아웃바운드 트래픽 감사	OTel exporter → 사내 SIEM	Telemetry
DLP (민감 데이터 차단)	PreToolUse Hook + dlp-check.sh	Hook 스크립트
사용자 활동 감사	CloudTrail (Amazon Bedrock) 또는 SSO 로그	Cloud 로그

6 areas

매핑 영역

Standard

사내 정책 그대로

Telemetry

감사 통합

Hook

강제 적용

네트워크 설정 체크리스트

배포 전 8가지 확인 사항

No	Item	담당
1	HTTP_PROXY, HTTPS_PROXY, NO_PROXY 표준화	IT
2	사내 CA 인증서 PC에 배포	IT
3	아웃바운드 방화벽 화이트리스트 추가	Sec
4	VPC Endpoint 설정 (Bedrock 사용 시)	Cloud
5	사내 DNS 분기 설정 (필요 시)	IT
6	VPN 또는 ZTNA 클라이언트 사전 설치	IT
7	DLP Hook 스크립트 배포	Sec
8	doctor 명령으로 모든 항목 검증	사용자

8 items
체크리스트

IT/Sec/Cloud
담당 분리

doctor
최종 검증

Pre-flight
배포 전 필수

Part 3 Recap

네트워크와 보안 핵심 정리

TAKEAWAY 1

프록시 통과

HTTP/HTTPS_PROXY +
NO_PROXY 표준화,
GPO/Ansible로 일괄 배포

TAKEAWAY 2

인증서 관리

NODE_EXTRA_CA_CERTS
+ 시스템 trust store, TLS
검증 절대 비활성 X

TAKEAWAY 3

도메인 통제

6개 필수 도메인 화이트리
스트, 사내 DNS 분기, VPC
Endpoint 사설

TAKEAWAY 4

DLP와 감사

PreToolUse Hook으로 민감
정보 차단, OTEL로 SIEM 통합

NEXT - PART 4

Part 4는 거버넌스와 정책입니다. managed-settings.json, 권한 강제, 조직 정책 계층, MCP 통제를 15장에 걸쳐 다룹니다.

Part 3

완료

Part 4

Governance (15p)

Part 5

Monitoring (15p)

~2.5h

잔여 시간

CHAPTER 03 - PART 04

Admin Setup – 거버넌스와 정책

Enterprise 배포와 거버넌스



Part 4 / 거버넌스와 정책

조직 차원의 통제 강제

PART OBJECTIVE

managed-settings.json을 통한 조직 정책 강제, 권한 hierarchy 이해, MCP 서버 통제, Hooks 활용까지 거버넌스 구축의 모든 측면을 다룹니다.

TOPIC 1

Settings 계층

managed-settings, project, user 3단 계층과 우선순위를 이해합니다.

TOPIC 2

권한 강제

allow/ask/deny 규칙, 도 구별 그룹 패턴, 명령 화이트 리스트를 설정합니다.

TOPIC 3

조직 정책

모든 사용자에게 강제되는 정책을 managed-settings로 배포합니다.

TOPIC 4

MCP 통제

허용 MCP 서버 명시, 위험 서버 차단, 사내 서버 우선 등록을 다룹니다.

15 Slides

Part 4 분량

~30 min

예상 강의 시간

3-tier

Settings 계층

Enforce

강제 적용

Settings 계층 구조

3단 계층의 우선순위

Hierarchy

```
# 우선순위 (높음 → 낮음)

# 1. managed-settings.json (조직 강제, 최우선)
# 위치 OS별:
#   macOS: /Library/Application Support/Claude/managed-settings.json
#   Linux:  /etc/claude/managed-settings.json
#   Windows: C:\ProgramData\Claude\managed-settings.json
# 권한: 일반 사용자가 수정 불가 (관리자 권한 필요)

# 2. project settings (.claude/settings.json)
# 위치: 프로젝트 루트의 .claude/settings.json
# 권한: 프로젝트 팀이 Git으로 관리
# 사용: 프로젝트별 특수 설정

# 3. user settings (~/.claude/settings.json)
# 위치: 사용자 홈
# 권한: 본인만 수정
# 사용: 개인 선호 (테마, 단축키 등)

# 병합 규칙
- deny: 어느 계층이든 deny면 차단 (deny 항상 승리)
- allow/ask: 더 상위 계층(managed > project > user) 우선
```

managed
조직 (최우선)

project
팀 공유

user
개인

deny wins
차단 우선

managed-settings.json 예시

조직 표준 정책

managed-settings

```
# /etc/claude/managed-settings.json
{
  "permissions": {
    "allow": [
      "Read(**)",
      "Grep", "Glob",
      "Bash(npm test:*)", "Bash(npm run lint)",
      "Bash(git diff:*)", "Bash(git status)", "Bash(git log:*)"
    ],
    "ask": [
      "Edit(**)", "Write(**)",
      "Bash(git push:*)", "Bash(git commit:*)"
    ],
    "deny": [
      "Bash(rm -rf:*)",
      "Bash(curl:*)", "Bash(wget:*)",
      "Bash(* > /etc/*)",
      "Read(**/.env*)", "Read(**/secrets/**)"
    ]
  },

```

```
"model": {
  "allowedModels": ["sonnet", "haiku"],
  "deniedModels": ["opus"]
},
"updateChecker": { "enabled": false },
"telemetry": {
  "enabled": true,
  "endpoint": "https://otel.company.com/v1/traces"
}
}
```

allow

자동 허용

ask

사용자 확인

deny

강제 차단

telemetry

감사 강제

권한 패턴 작성법

그룹과 정규식 활용

Patterns

1. 도구 이름만 (전체 허용)

"Read", "Grep", "Glob"

2. 경로 그룹 패턴

"Read(src/**)"

src 디렉토리만 읽기 가능

"Edit(docs/**)"

docs만 편집 가능

"Read(**/*.md)"

모든 .md 파일만 읽기

3. Bash 명령 패턴

"Bash(npm test:*)"

npm test 시작하는 모든 명령

"Bash(git diff:*)"

git diff 변형들

"Bash(npm run lint)"

정확히 이 명령만

4. 부정 (deny에서만 의미)

"Read(**/.env*)"

.env 파일 차단

"Read(**/credentials/**)"

credentials 디렉토리 전체 차단

"Bash(*sudo*)"

sudo 포함 명령 모두 차단

5. 와일드카드 주의

"Bash(*)" # 모든 Bash 허용 (위험)

"Edit(*)" # 모든 Edit 허용 (주의)

6. 우선순위

더 구체적인 패턴이 일반 패턴보다 우선

allow Bash(npm test:*) + deny Bash(npm:*) → npm test만 허용

Glob

경로 패턴

Bash():*

명령 패턴

Specific

구체 우선

Test thoroughly

검증 필수

조직 정책 설계 전략

안전과 생산성의 균형

LEVEL 1

읽기 중심 (보수적)

Read/Grep/Glob 허용,
Edit/Write는 ask, Bash 대
부분 deny.
코드 분석 위주.

LEVEL 2

제한적 쓰기 (표준)

Edit 허용, 안전한 Bash 자
동, 위험 명령 deny.
일반 개발에 적합.

LEVEL 3

광범위 자율 (개발자)

Edit/Write/Bash 광범위
허용.
시니어 개발자나 신뢰된 사
용자.

LEVEL 4

CI/CD (자동화)

특정 명령만 허용, 나머지
deny.
결정적 자동화 환경.

STRATEGY

사용자 그룹별로 다른 managed-settings를 GPO/MDM으로 배포.

일반 직원은 LEVEL 2, 시니어는 LEVEL 3, CI는 LEVEL 4 같은 식으로 차등 적용.

L1

보수

L2

표준

L3

자율

L4

CI

managed-settings 배포

GPO와 MDM으로 일괄 적용

Distribution

```
# 1. Windows GPO (도메인 환경)
# Group Policy Editor:
# Computer Configuration → Preferences →
#   Windows Settings → Files
# Source: \\fileserver\Configs\managed-settings-L2.json
# Destination: C:\ProgramData\Claude\managed-settings.json
# Action: Replace
# Targeting: 개발팀 그룹
```

```
# 2. macOS Jamf Configuration Profile
# Custom Settings Payload
# Preference Domain: com.anthropic.claude
# Property List: managed-settings.json 내용
```

```
# 3. Linux Ansible
- name: Deploy managed-settings
  hosts: developers
  become: yes
  tasks:
    - name: Copy config
      copy:
        src: managed-settings-L2.json
        dest: /etc/claude/managed-settings.json
        owner: root
        mode: '0644'
    - name: Verify
      command: claude doctor
```

```
# 4. 그룹별 다른 파일 배포 가능
# L1: managed-settings-readonly.json
# L2: managed-settings-standard.json
# L3: managed-settings-developer.json
```

GPO

Windows

Jamf

macOS

Ansible

Linux

Per-group

그룹별 차등

MCP 서버 화이트리스트

허용 MCP만 사용 가능

MCP Policy

```
# managed-settings.json에 MCP 통제 추가
{
  "mcpPolicy": {
    "mode": "allowlist",
    "allowedServers": [
      "jira-internal",
      "confluence-internal",
      "gitlab-internal",
      "filesystem",
      "github"
    ],
    "deniedServers": [
      "any-untrusted-server"
    ],
    "requireApproval": [
      "shell",
      "browser"
    ]
  }
}
```

```
# 효과
- 사용자가 .claude/settings.json에 추가한
  MCP 서버 중 allowlist에 없으면 무시
- deniedServers에 명시된 서버는 차단
- requireApproval은 매번 사용자 확인

# 사내 MCP 서버 사전 등록 (자동 설정)
{
  "mcpServers": { ... 사내 표준 서버들 ... }
}
```

이를 managed-settings에 함께 포함하면 모든 PC에 자동 등록

allowlist

명시 허용

deny

명시 차단

requireApproval

매번 확인

Auto-register

사내 자동 등록

Hooks로 거버넌스 강제

정책을 코드로 강제하기

Mandatory Hooks

```
# managed-settings.json
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [{
          "type": "command",
          "command": "/opt/claude/hooks/audit-log.sh"
        }]
      },
      {
        "matcher": "Edit|Write",
        "hooks": [{
          "type": "command",
          "command": "/opt/claude/hooks/dlp-check.sh"
        }]
      }
    ]
  },
}
```

```
"SessionStart": [
  {
    "hooks": [{
      "type": "command",
      "command": "/opt/claude/hooks/policy-banner.sh"
    }]
  }
]
```

효과

- 모든 Bash 호출이 사내 SIEM에 로깅
- 모든 Edit/Write가 DLP 검사 통과 필요
- 세션 시작 시 사용 정책 안내 메시지

PreToolUse

도구 호출 전

SessionStart

세션 시작

PostToolUse

도구 호출 후

Stop

응답 종료 시

Audit Log Hook 예시

사내 SIEM 통합

```
#!/bin/bash
# /opt/claude/hooks/audit-log.sh
INPUT=$(cat) # stdin으로 도구 호출 정보
# 사용자 정보 수집
USER=$(whoami)
HOST=$(hostname)
TS=$(date -u +%Y-%m-%dT%H:%M:%SZ)
SESSION_ID=$(echo "$INPUT" | jq -r '.session_id')
TOOL=$(echo "$INPUT" | jq -r '.tool_name')
TOOL_INPUT=$(echo "$INPUT" | jq -r '.tool_input')
```

```
# JSON 형식으로 SIEM 페이로드 작성
PAYLOAD=$(cat <<EOF
{
  "timestamp": "$TS",
  "user": "$USER",
  "host": "$HOST",
  "session_id": "$SESSION_ID",
  "tool": "$TOOL",
  "tool_input_hash": "$(echo $TOOL_INPUT | sha256sum | cut -d' ' -f1)",
  "app": "claude-code"
}
EOF
)
# 사내 SIEM API로 전송 (Splunk, ELK, Datadog 등)
curl -s -X POST https://siem.company.com/api/log \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $SIEM_TOKEN" \
  -d "$PAYLOAD" > /dev/null
# 통과 (도구 호출 진행 허용)
exit 0
```

audit-log.sh

jq
JSON 파싱

sha256
본문 해시

SIEM API
사내 전송

Non-blocking
도구 진행

Policy Banner Hook

세션 시작 시 정책 안내

```
#!/bin/bash
# /opt/claude/hooks/policy-banner.sh
```

```
cat <<EOF
```

Claude Code - Company Internal Use

- ✓ 본인 활동은 사내 SIEM에 기록됩니다
- ✓ 민감 정보 (Key, 비밀번호) 입력은 자동 차단됩니다
- ✓ 비용 한도: 월 \$50/인 (초과 시 사용 제한)
- ✓ 정책 위반 시 IT 보안팀 연락: security@company.com

허용 모델: Sonnet, Haiku / Opus: 별도 신청 필요
허용 MCP: jira, confluence, gitlab, github

```
EOF
```

policy-banner.sh

```
# 매 세션 시작 시 정책 안내
# 사용자가 이미 알고 있어도 매번 표시 (의식 제고)
exit 0
```

PURPOSE

정책 배너는 사용자의 정책 인식을 제고하고, 사고 발생 시 "몰랐다"는 변명을 차단하는 컴플라이언스 효과가 있음.

SessionStart

실행 시점

Inform

안내 목적

Compliance

컴플라이언스

Every session

매 세션

일반적 정책 위반과 대응

실제로 발생하는 문제

위반	발생 시점	대응
API Key를 코드에 하드코딩	Edit/Write 시	DLP Hook 차단
rm -rf 같은 위험 명령	Bash 호출	deny 규칙 차단
Opus 무단 사용	세션 시작 시 모델 선택	deniedModels 차단
비승인 MCP 서버 등록	settings.json 수정 시	mcpPolicy 무시
월 한도 초과 사용	누적 사용	예산 알람 + 일시 제한
외부 도메인 접근	Bash curl/wget	방화벽 차단

6 cases

대표 위반

Auto-block

자동 차단

Alert

알림 발송

Review

주기 검토

정책 업데이트 워크플로우

변경 시 안전한 적용

STEP 01

정책 변경
제안 작성

STEP 02

검토와
승인 절차

STEP 03

Lab 검증
(1-2일)

STEP 04

Canary
그룹 배포

STEP 05

전체 배포
+ 모니터

CHANGE MANAGEMENT

정책 변경은 코드와 동일하게 Git으로 버전 관리. PR 리뷰, Lab 검증, Canary 배포, 전체 배포의 단계를 거쳐 안전하게 롤아웃.

Git

버전 관리

PR review

변경 검토

Staged

단계 배포

Rollback

즉시 복귀

사용자 경험 고려

엄격함과 생산성의 균형

TOO STRICT

너무 엄격한 경우

1. ask 규칙이 과다해 매번 확인
2. 권한 부족 에러 빈발
3. 생산성 저하, 우회 시도 유발
4. 결국 사용자가 외면
5. 도입 실패 가능성

BALANCED

균형 잡힌 정책

1. 명백히 위험만 deny
2. 일반적 작업은 allow
3. 그레이존만 ask
4. 사용자 친화적 에러 메시지
5. 우회 욕구 없는 자연스러운 사용

Trust

사용자 신뢰

Sensible

합리적 통제

Feedback

사용자 피드백

Iterate

지속 개선

정책 효과 측정

KPI와 모니터링 지표

KPI 1

위반 차단 횟수

deny 규칙으로 실제 차단된 호출 수. 너무 많으면 정책이 과도하거나 사용자 교육 부족.

KPI 2

ask 승인율

ask 규칙에서 사용자 승인 비율. 항상 승인이면 ask → allow 전환 검토.

KPI 3

평균 세션 시간

세션 시간이 줄면 정책 마찰 가능성. 사용자 인터뷰로 원인 파악.

KPI 4

도입률

대상 사용자 중 실제 활용률. 낮으면 도구 또는 정책 문제일 가능성.

REVIEW CYCLE

KPI는 주간 또는 월간으로 검토하며 정책 개선 사이클로 연결. 사용자 피드백과 함께 데이터 기반 결정을 함.

4 KPIs

핵심 지표

Weekly

검토 주기

Data-driven

데이터 기반

Iterate

정책 개선

Part 4 Recap

거버넌스와 정책 핵심 정리

TAKEAWAY 1

3단 계층

managed-settings >
project > user, deny는
항상 승리

TAKEAWAY 2

4단계 정책 레벨

L1 보수 / L2 표준 / L3 자율
/ L4 CI, 그룹별 차등 적용

TAKEAWAY 3

Hooks 강제

PreToolUse 후크로 DLP
검사, SessionStart 후크로
정책 안내

TAKEAWAY 4

균형이 핵심

너무 엄격하면 우회 시도,
KPI 기반 지속 개선 필수

NEXT - PART 5

Part 5는 모니터링과 비용 통제입니다. OpenTelemetry, /cost 명령, 사내 SIEM 통합, 예산 알람을 15장에 걸쳐 다룹니다.

Part 4

완료

Part 5

Monitoring (15p)

Part 6

SSO (15p)

~2h

잔여 시간

CHAPTER 03 - PART 05

Admin Setup – 모니터링과 비용통제

Enterprise 배포와 거버넌스



© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved. Amazon Confidential and Trademark.

Part 5 / 모니터링과 비용 통제

운영 가시성과 예산 통제

PART OBJECTIVE

OpenTelemetry exporter를 통한 사내 SIEM 통합, /cost 명령, CloudWatch와 Cost Explorer를 활용한 비용 추적, 예산 알람 자동화까지 모니터링의 모든 측면을 다룹니다.

TOPIC 1

Telemetry 기본

OpenTelemetry 표준, exporter 설정, 사내 SIEM과의 통합 패턴을 다룹니다.

TOPIC 2

비용 추적

/cost 명령, JSON 출력, 사용자별 비용 분리, 모델별 추적을 학습합니다.

TOPIC 3

Cloud 통합

CloudWatch dashboard, Cost Explorer, GCP Billing 통합을 다룹니다.

TOPIC 4

예산 알람

월 한도 알람, 임계치 액션, 자동 차단, 알림 채널 통합을 구축합니다.

15 Slides

Part 5 분량

~30 min

예상 강의 시간

OTel

표준 프로토콜

Visibility

핵심 목표

OpenTelemetry 개요

관측 가능성의 표준

OTEL

OpenTelemetry는 메트릭, 트레이스, 로그를 표준 형식으로 수집하고 전송하는 오픈 표준.

Claude Code는 OTLP 프로토콜로 직접 exporter를 지원.

METRICS

메트릭

토큰 수, API 호출, 응답 시간, 모델 사용량, 도구 호출 횟수 등 수치 데이터.

TRACES

트레이스

세션 시작부터 종료까지의 흐름, 도구 호출 chain, Subagent 위임 경로를 추적.

LOGS

로그

에러, 경고, 정책 위반 같은 이벤트 기록. 텍스트 또는 구조화 JSON 형식.

OTLP

표준 프로토콜

3 pillars

Metrics/Traces/Logs

Vendor-neutral

벤더 중립

Native

Claude Code 내장

OTel 설정

managed-settings를 통한 강제

OTel config

```
# managed-settings.json에 telemetry 섹션 추가
{
  "telemetry": {
    "enabled": true,
    "endpoint": "https://otel-collector.company.com:4318",
    "headers": {
      "Authorization": "Bearer ${OTEL_TOKEN}"
    },
  },
  "exporters": ["traces", "metrics", "logs"],
  "sampling": {
    "traces": 1.0,
    "metrics": "always"
  },
  "attributes": {
    "service.name": "claude-code",
    "deployment.environment": "production",
    "company.region": "kr"
  }
}
```

효과

- 모든 Claude Code 인스턴스가 사내 OTel Collector로 전송
- 사용자가 비활성화할 수 없음 (managed 강제)
- 표준 OTLP 프로토콜로 호환성 보장
- 사내 SIEM/APM과 자동 통합

4318

OTLP HTTP 포트

Bearer

인증 헤더

Sampling

샘플링 비율

Attributes

메타데이터

SIEM 통합

주요 SIEM 솔루션 연동

SIEM	연동 방식	데이터 형식
Splunk	OTel Collector → Splunk HEC	OTLP → Splunk JSON
Datadog	Datadog Agent로 OTLP 수신	OTLP → Datadog
ELK Stack	OTel Collector → Logstash 또는 Elasticsearch	OTLP → Elasticsearch
Grafana Stack	OTel → Tempo (traces) + Mimir (metrics) + Loki (logs)	OTLP 직접
AWS CloudWatch	CloudWatch Agent OTLP receiver	OTLP → CloudWatch

COLLECTOR PATTERN

OTel Collector를 사내에 둔 후 그 뒤에 SIEM 솔루션을 두는 패턴이 표준. Collector가 형식 변환과 라우팅을 담당해 SIEM 변경이 유연.

Splunk
HEC 호환

Datadog
Agent 수신

ELK
Logstash

CloudWatch
AWS 통합

/cost 명령

현재 세션 비용 즉시 확인

/cost

1. 대화형 사용

```
$ claude  
> /cost
```

Session Cost Summary

Session ID: ses_abc123

Started: 2026-05-25 09:15:30 KST

Duration: 2h 14m

Cost breakdown:

Main agent \$0.82 (1,250 input + 320 output tokens, Sonnet)

security-scanner \$1.05 (Opus, 850 tokens output)

test-writer \$0.41 (Sonnet, 920 tokens output)

docs-writer \$0.19 (Sonnet, 410 tokens output)

Total this session: \$2.47

Daily cumulative: \$12.83

Monthly cumulative: \$87.42 / \$200 limit (44%)

2. 헤드리스 + JSON 출력

```
$ claude -p "..." --output-format json | jq '.usage'  
{  
  "input_tokens": 5200,  
  "output_tokens": 850,  
  "model": "claude-sonnet-4-5",  
  "estimated_cost_usd": 0.0285  
}
```

3. 모니터링 자동화에 활용

/cost

대화형 명령

JSON

헤드리스 출력

Cumulative

누적 추적

Limit

한도 표시

CloudWatch 대시보드

AWS Bedrock 사용 시 비용 가시화

CloudFormation으로 대시보드 자동 생성

Resources:

ClaudeCodeDashboard:

Type: AWS::CloudWatch::Dashboard

Properties:

DashboardName: ClaudeCode-Operations

DashboardBody: !Sub |

```
{
  "widgets": [
    {
      "type": "metric",
      "properties": {
        "metrics": [
          ["AWS/Bedrock", "Invocations",
            "ModelId", "anthropic.claude-sonnet-4-5"],
          ["...", "InputTokenCount"],
          ["...", "OutputTokenCount"]
        ],
        "stat": "Sum",
        "period": 300,
        "title": "Sonnet Usage (5m)"
      }
    }
  ],
}
```

```
{
  "type": "metric",
  "properties": {
    "metrics": [["AWS/Bedrock", "InvocationLatency"]],
    "stat": "Average",
    "title": "Latency p50/p99"
  }
}
```

CW Dashboard

Bedrock metrics

자동 수집

Invocations

호출 횟수

Tokens

입출력 토큰

Latency

응답 시간

AWS Cost Explorer

월간 비용 분석

Cost Explorer

```
# 1. Cost Explorer에서 Bedrock 비용 필터링
# AWS Console → Cost Explorer
# Filter: Service = Amazon Bedrock
```

```
# 2. 태그 기반 비용 분리
# IAM Role에 Cost Allocation Tag 설정:
#   Team: backend
#   Project: payment-service
#   Environment: production
```

```
# CLI로 태그 기반 비용 조회
aws ce get-cost-and-usage \
  --time-period Start=2026-05-01,End=2026-05-31 \
  --granularity DAILY \
  --metrics UnblendedCost \
  --group-by Type=TAG,Key=Team \
  --filter file://filter.json
```

```
# filter.json
{
  "Dimensions": {
    "Key": "SERVICE",
    "Values": ["Amazon Bedrock"]
  }
}
```

```
# 3. 결과 (예시)
DAILY Costs by Team (May)
Backend Team:  $245.30
Frontend Team: $118.50
Data Team:     $87.20
Total:         $451.00
```

Tag-based

태그 분리

DAILY

일별 추적

aws ce

CLI 자동화

Chargeback

부서 청구

예산 알람 설정

한도 초과 자동 대응

AWS Budgets

```
# AWS Budgets로 자동 알람 설정
aws budgets create-budget \
  --account-id 123456789012 \
  --budget '{
    "BudgetName": "ClaudeCode-Monthly",
    "BudgetLimit": { "Amount": "500", "Unit": "USD" },
    "TimeUnit": "MONTHLY",
    "BudgetType": "COST",
    "CostFilters": {
      "Service": ["Amazon Bedrock"]
    }
  }' \
```

```
--notifications-with-subscribers '[
  {
    "Notification": {
      "NotificationType": "ACTUAL",
      "ComparisonOperator": "GREATER_THAN",
      "Threshold": 80,
      "ThresholdType": "PERCENTAGE"
    },
    "Subscribers": [
      { "SubscriptionType": "EMAIL",
        "Address": "admin@company.com" },
      { "SubscriptionType": "SNS",
        "Address": "arn:aws:sns:...:claude-alerts" }
    ]
  }
]'
```

```
# 다단계 임계치
# 50% / 80% / 100% / 120% 별로 다른 액션
```

\$500/mo

월 한도

80%

경고 임계치

Email+SNS

알림 채널

Multi-tier

다단계

자동 사용량 제한

한도 초과 시 자동 차단

Auto throttle

```
# 1. SNS topic 구독자에 Lambda 함수 등록  
# Budget 100% 도달 → SNS → Lambda → IAM 정책 수정
```

```
# Lambda 함수 (Python)  
import boto3, json
```

```
def lambda_handler(event, context):  
    iam = boto3.client('iam')
```

```
    # 사용자 그룹에 Bedrock deny 정책 부착  
    iam.attach_group_policy(  
        GroupName='claude-code-users',  
        PolicyArn='arn:aws:iam::123:policy/BedrockTempDeny'  
    )
```

```
    # Slack/Teams 알림  
    notify_slack({  
        'channel': '#claude-alerts',  
        'text': '🔴 월 예산 100% 초과! 사용 일시 중지됨'  
    })
```

```
    return {'status': 'throttled'}
```

```
# BedrockTempDeny 정책
```

```
{  
    "Effect": "Deny",  
    "Action": ["bedrock:InvokeModel"],  
    "Resource": "*"   
}
```

```
# 다음 달 1일 자동 해제 (EventBridge schedule)
```

```
# 또는 Admin 수동 해제
```

Lambda

액션 처리

IAM Deny

강제 차단

Slack

알림

Auto-reset

월초 자동 해제

사용자별 비용 추적

개인별 사용량 모니터링

Per-user

1. IAM Identity Center 사용자별 추적 (Bedrock)

CloudTrail이 자동으로 사용자 신원 기록

userIdentity.principalId에서 사용자 식별

2. CloudWatch Insights 쿼리

```
fields @timestamp, userIdentity.principalId, modelId,
       inputTokens, outputTokens
| filter eventSource = "bedrock.amazonaws.com"
| stats sum(inputTokens) as totalIn,
         sum(outputTokens) as totalOut by principalId
| sort totalOut desc
| limit 50
```

3. 결과 (예시)

principalId	totalIn	totalOut
alice@company.com	2,450,000	180,000
bob@company.com	890,000	120,000
charlie@company.com	620,000	45,000

4. 월간 자동 리포트

- Lambda + EventBridge 월 1일 트리거
- 위 쿼리 실행
- 사용자별 PDF 리포트 생성
- email/Slack DM 발송

5. 개인별 한도 (선택)

- IAM Role 정책에 사용자별 사용량 attribute
- 한도 초과 시 자동 ask 모드 전환

principalId

사용자 식별

CW Insights

쿼리 도구

Monthly

월간 리포트

Per-user

개인 한도

비용 최적화 전략

5가지 실전 절감 방법

STRATEGY 1

Haiku 적극 활용

단순 분류, 추출, 변환 작업은 Haiku로. Sonnet 대비 67% 절감 가능합니다.

STRATEGY 2

컨텍스트 관리

/clear와 /compact를 적극 활용하여 불필요한 토큰 누적을 막습니다.

STRATEGY 3

Plan 모드

복잡 작업 전 Plan 모드로 의도를 명확히 하여 시행착오를 줄입니다.

STRATEGY 4

Opus 신중 사용

심층 분석에만 Opus, 일반 작업은 Sonnet. 별도 신청 절차로 통제합니다.

STRATEGY 5 / 캐싱 활용

Prompt caching 기능을 활용해 반복 컨텍스트의 비용을 90%까지 절감할 수 있음. 큰 CLAUDE.md나 코드베이스 컨텍스트에 효과적.

-67%

Haiku

-30%

컨텍스트 관리

-20%

Plan 모드

-90%

Prompt cache

토큰 효율성 팁

작은 변화 큰 효과

Token tips

```
# 1. CLAUDE.md를 간결하게
# ❌ 길고 모호한 설명
"""## Architecture
This project follows clean architecture principles with
domain layer, application layer, and infrastructure layer...""" # 500+ tokens

# ✅ 핵심만 간결하게
"""## Architecture
3-layer: domain (src/domain), app (src/app), infra (src/infra)""" # 30 tokens

# 2. 큰 파일은 부분 읽기
# ❌ 전체 파일 읽고 분석 (10K tokens)
Read("src/giant-file.ts")

# ✅ 필요한 부분만
Read("src/giant-file.ts", { offset: 100, limit: 50 })

# 3. Grep으로 미리 좁히기
# 먼저 Grep으로 관련 코드 위치 파악 (적은 토큰)
# 그 후 해당 파일/라인만 Read

# 4. Subagent로 큰 분석 위임
# 큰 분석을 메인이 직접 하면 컨텍스트 폭발
# Subagent에 위임하면 결과만 받음 (90%+ 절감)

# 5. 출력 형식 명시
# JSON 스키마를 미리 지시하여 불필요한 설명 제거
```

Concise

간결한 CLAUDE.md

Partial Read

부분 읽기

Grep first

검색 우선

Delegate

서브 위임

이상 사용 감지

비정상 패턴 자동 탐지

Pattern	의심 사유	대응
24시간 무중단 사용	자동화 또는 봇 의심	계정 조사
갑작스러운 토큰 폭증	대량 데이터 분석 또는 사고	Slack 알림
Opus 비정상 빈도	단순 작업에 비싼 모델 사용	사용자 교육
외부 도메인 다수 호출	데이터 유출 가능성	즉시 차단
비정상 시간대 활동	계정 탈취 가능성	MFA 재인증

SIEM

탐지 위치

Baseline

정상 기준선

Alert

자동 알림

Investigate

조사 절차

분기별 운영 리뷰

데이터 기반 개선 사이클

STEP 01

데이터 수집
3개월

STEP 02

KPI 분석
FinOps

STEP 03

개선 안건
도출

STEP 04

정책/도구
조정

STEP 05

다음 분기
실행

METRIC 1

비용 추세

월간 비용 증감, 사용자당 비용, 모델별 분포

METRIC 2

도입률

활성 사용자 비율, 세션 수, 작업 다양성

METRIC 3

품질 지표

에러율, 거부된 호출 비율, 사용자 만족도

METRIC 4

보안 이벤트

DLP 차단, 정책 위반, 이상 패턴 발생

Quarterly
주기

4 metrics
리뷰 지표

FinOps
재무 협업

Iterate
지속 개선

Part 5 Recap

모니터링과 비용 통제 핵심 정리

TAKEAWAY 1

OpenTelemetry

managed-settings로 강제 활성화, OTel Collector 패턴으로 SIEM 통합

TAKEAWAY 2

비용 가시화

/cost 명령, CloudWatch 대시보드, 사용자별 추적으로 chargeback

TAKEAWAY 3

예산 통제

다단계 임계치 + Lambda 자동 차단 + Slack 알림

TAKEAWAY 4

최적화 5선

Haiku, /clear, Plan, Opus 신중, Prompt cache

NEXT - PART 6

Part 6은 SSO와 신원 통합입니다. IAM Identity Center 심화, Okta, Azure AD, SAML, OIDC, group mapping을 15장에 걸쳐 다룹니다.

Part 5

완료

Part 6

SSO (15p)

Part 7-9

Audit + Labs (25p)

~1.5h

잔여 시간

CHAPTER 03 - PART 06

Admin Setup – SSO와 신원통합

Enterprise 배포와 거버넌스



Part 6 / SSO와 신원 통합

엔터프라이즈 신원 시스템 연동

PART OBJECTIVE

IAM Identity Center 심화, Okta/Azure AD/Google Workspace 같은 IdP와의 SAML 또는 OIDC 통합, 그룹 매핑, MFA 강제, JIT 프로비저닝까지 신원 관리 전반을 다룹니다.

TOPIC 1

IdP 옵션

Okta, Azure AD, Google Workspace, IAM Identity Center 4대 IdP 비교.

TOPIC 2

SAML vs OIDC

두 프로토콜의 차이, 사용 시나리오, 구현 방법을 다룹니다.

TOPIC 3

Group Mapping

IdP 그룹을 AWS Role 또는 Claude Code 권한 그룹으로 매핑합니다.

TOPIC 4

MFA & JIT

다중 인증 강제, Just-In-Time 프로비저닝 자동화 패턴을 학습합니다.

15 Slides

Part 6 분량

~30 min

예상 강의 시간

4 IdPs

주요 솔루션

Enterprise

필수 영역

IdP 4대 솔루션 비교

조직 환경별 선택

IdP	강점	적합 환경
AWS IAM Identity Center	AWS 통합 최강, 무료, Bedrock 직접 연동	AWS 중심 조직
Okta	IdP 시장 선도, 7K+ 앱 통합, 풍부한 정책	멀티 클라우드
Azure AD (Entra ID)	Microsoft 365 통합, AD 연계, 친숙	M365 사용 조직
Google Workspace	Google 생태계, 간단한 설정	GCP 또는 Workspace 사용

RECOMMENDATION

이미 IdP가 있으면 그것을 사용. 새로 도입한다면 AWS 중심은 IAM IC, 멀티 클라우드는 Okta, M365는 Azure AD를 추천.

IAM IC

AWS

Okta

범용

Entra

Microsoft

Workspace

Google

SAML vs OIDC

두 프로토콜의 비교

SAML

Security Assertion Markup Language

1. XML 기반 (오래된 표준)
2. 엔터프라이즈 SSO 표준
3. 브라우저 리디렉션 흐름
4. AWS IAM, Salesforce 표준
5. 복잡한 설정
6. 모든 IdP가 지원

OIDC

OpenID Connect

1. OAuth 2.0 위에 구축
2. JSON 기반 (현대 표준)
3. 모바일과 API 친화적
4. 간단한 토큰 흐름
5. 빠른 통합 가능
6. 대부분 IdP 지원

SAML

XML / 엔터프라이즈

OIDC

JSON / 현대

Both

두 옵션 가능

IdP-dependent

IdP 선택 좌우

IAM Identity Center 심화

AWS 신원 통합 표준

IAM IC

```
# 1. 활성화 (한 번만)
# AWS Console → IAM Identity Center → Enable
# 자동으로 .awsapps.com URL 생성
```

```
# 2. ID source 설정
# 옵션 A: Identity Center 디렉토리 (내장)
# 옵션 B: 외부 IdP 연동 (SAML)
#   - Okta, Azure AD, Google 등 연결
#   - SCIM으로 사용자 자동 프로비저닝
```

```
# 3. Permission Set 생성 (역할 묶음)
# PowerUser-ClaudeCode:
#   - bedrock-claude-code-policy
#   - 세션 시간: 8h
#   - 인라인 정책 또는 관리형 정책 첨부
```

```
# 4. AWS 계정에 어사인먼트
# Group: developers → PowerUser-ClaudeCode → Account: prod
# Group: contractors → ReadOnly-ClaudeCode → Account: dev
```

```
# 5. 사용자 사용 흐름
$ aws sso login --profile claude
# 브라우저 자동 열림 → SSO 로그인 → 임시 자격증명
$ claude # Bedrock 자동 연결
```

Free

AWS 무료 서비스

SCIM

자동 프로비저닝

Permission Set

권한 묶음

SSO login

단일 로그인

Okta 연동

IdP 시장 선도 솔루션

Okta + AWS

1. Okta Admin Console에서 SAML App 생성
Applications → Add Application → AWS Account Federation
또는: Create App Integration → SAML 2.0

2. SAML 설정
SSO URL: <https://signin.aws.amazon.com/saml>
Audience URI: [urn:amazon:webservices](https://signin.aws.amazon.com/saml)
Name ID Format: EmailAddress

Attribute Statements
- <https://aws.amazon.com/SAML/Attributes/Role>
 = [arn:aws:iam::123:role/ClaudeCodeRole](#),
 [arn:aws:iam::123:saml-provider/Okta](#)
- <https://aws.amazon.com/SAML/Attributes/RoleSessionName>
 = user.email

3. Group Mapping (Okta groups → AWS Roles)
- okta_developers → ClaudeCodeRole (allows Sonnet, Haiku)
- okta_seniors → ClaudeCodeAdvancedRole (allows all models)

4. 사용자 흐름
1. <https://company.okta.com> 로그인
2. AWS Account Federation 클릭
3. 자동으로 AWS Console 또는 CLI 자격증명 발급
4. Claude Code 실행 → Bedrock 자동 사용

5. saml2aws CLI 도구 활용 (옵션)

SAML 2.0

프로토콜

Federation

AWS 연동

Group → Role

매핑

saml2aws

CLI 도구

Group Mapping 전략

권한 부여의 핵심

IdP Group	AWS Role	권한
engineering-juniors	ClaudeCodeBasic	Read + Sonnet only
engineering-seniors	ClaudeCodeAdvanced	All models + Edit
data-scientists	ClaudeCodeDataSci	Opus + Jupyter MCP
qa-engineers	ClaudeCodeQA	Test 도구 + Read
contractors	ClaudeCodeReadOnly	Read only + Haiku
admin	ClaudeCodeAdmin	전체 + Opus + 관리

Group-based

그룹 기반

Least-priv

최소 권한

Role separation

역할 분리

Promotion path

승급 경로

MFA 강제 적용

다중 인증 필수화

MFA Policy

```
# 1. IdP 측 MFA 강제 (가장 강력)
# Okta: Policy → Authentication → MFA
# Azure AD: Conditional Access → Require MFA
# Google: Admin → 2-Step Verification → Enforce
```

```
# 2. AWS Role Trust Policy에 MFA 조건
{
  "Effect": "Allow",
  "Principal": {
    "Federated": "arn:aws:iam::123:saml-provider/Okta"
  },
  "Action": "sts:AssumeRoleWithSAML",
  "Condition": {
    "BoolIfExists": {
      "aws:MultiFactorAuthPresent": "true"
    }
  }
}
```

```
# 3. Bedrock 호출에 MFA 조건 (정책 수준)
{
  "Effect": "Deny",
  "Action": "bedrock:InvokeModel",
  "Resource": "*",
  "Condition": {
    "BoolIfExists": { "aws:MultiFactorAuthPresent": "false" }
  }
}
```

```
# 4. 이중 보호 효과
- IdP에서 1차 차단
- AWS에서 2차 확인
- MFA 없이는 절대 사용 불가
```

IdP-level

1차 강제

Trust policy

2차 확인

Action-level

3차 정책

Defense in depth

심층 방어

JIT 프로비저닝

Just-In-Time 자동 등록

STEP 01

사용자
첫 SSO 로그인

STEP 02

IdP가 SAML
응답 전송

STEP 03

AWS가 사용자
자동 생성

STEP 04

그룹 매핑
Role 부여

STEP 05

Bedrock
즉시 사용

BENEFIT

JIT 프로비저닝은 사용자를 AWS에 미리 등록할 필요 없이 IdP 권한만 부여. 신규 직원의 첫 로그인 시 자동으로 AWS 사용자가 생성.

```
# Trust Policy + Condition으로 그룹 기반 자동 매핑
{
  "Effect": "Allow",
  "Principal": { "Federated": "arn:aws:iam::123:saml-provider/Okta" },
  "Action": "sts:AssumeRoleWithSAML",
```

```
    "Condition": {
      "StringEquals": {
        "SAML:aud": "https://signin.aws.amazon.com/saml"
      },
      "ForAnyValue:StringLike": {
        "saml:groups": ["engineering-*"]
      }
    }
  }
}
```

JIT trust

No pre-reg

사전 등록 불필요

Auto-create

자동 생성

Group-based

그룹 기반

Day-one ready

첫날 사용

SCIM 프로비저닝

양방향 자동 동기화

SCIM

System for Cross-domain Identity Management는 사용자와 그룹 정보를 IdP와 AWS 사이에서 자동 동기화하는 표준 프로토콜.

```
# IAM Identity Center에서 SCIM 활성화
# Settings → Identity source → Configure SCIM
# - SCIM endpoint: https://scim.us-east-1.amazonaws.com/...
# - Access token: 시크릿 보관
```

```
# Okta SCIM 연결
# Applications → AWS SSO → Provisioning
# - Enable API integration
# - Base URL: SCIM endpoint
# - API Token: 위에서 받은 token
# - Test connection
```

```
# 자동 동기화되는 정보
- 사용자 생성/수정/삭제
- 그룹 멤버십 변경
- 사용자 속성 (email, name, dept)
```

```
# 효과
- 신입 직원 → IdP에 추가 → AWS에 자동 생성
- 퇴사자 → IdP에서 제거 → AWS에서 자동 비활성화
- 부서 이동 → IdP 그룹 변경 → AWS 권한 자동 변경
```

```
# 사람이 수동 동기화할 필요 없음
```

SCIM

Standard

SCIM 표준

Bidirectional

양방향

Real-time

실시간

Auto-cleanup

자동 정리

세션 관리

자격증명 수명 통제

Setting	권장 값	이유
IdP Session Duration	8시간	업무 시간
AWS STS Session	8시간 (max 12h)	IdP와 일치
Refresh Token	15분 자동 갱신	보안과 편의 균형
Idle Timeout	30분 무활동 시 만료	자리 이탈 방지
Concurrent Sessions	최대 3개	비정상 다중 로그인 차단

BEST PRACTICE

단기 세션 + 자동 갱신 패턴이 표준. 길게는 8시간을 넘기지 말고, 짧게는 사용자 경험을 해치지 않는 수준에서 균형을 잡습니다.

8h

권장 세션

15m

리프레시

30m

Idle

3

동시 세션

Offboarding 자동화

퇴사자 권한 즉시 회수

STEP 01

HR 시스템에서
퇴사 처리

STEP 02

IdP 사용자
비활성화

STEP 03

SCIM이 AWS에
전파

STEP 04

모든 세션
즉시 무효화

STEP 05

감사 로그
보관

CRITICAL

퇴사자 권한 회수는 보안 사고의 가장 흔한 원인입니다. HR → IdP → AWS의 자동 흐름이 사람의 실수를 막아 줍니다.

자동화 흐름 (예시)
HR System (Workday/SuccessFactors)
↓ webhook 또는 SCIM
IdP (Okta/Azure AD)
↓ deactivate user
↓ SCIM

AWS IAM Identity Center
↓ revoke active sessions
↓ disable user
CloudTrail에 모든 변경 기록

flow

HR Trigger

진실의 원천

Automated

사람 미개입

<5 min

회수 시간

Audit-ready

감사 가능

신원 이벤트 감사

누가 언제 무엇을 했는가

CloudTrail에서 자동 기록되는 신원 이벤트

- AssumeRoleWithSAML (SSO 로그인)
- AssumeRole (Role 전환)
- GetSessionToken (임시 자격증명 발급)
- ConsoleLogin (Console 직접 로그인)

Identity audit

이상 패턴 탐지 쿼리 (CloudWatch Insights)

```
fields @timestamp, eventName, userIdentity.principalId,
      sourceIPAddress, userAgent
| filter eventSource = "signin.amazonaws.com"
   or eventSource = "sts.amazonaws.com"
| filter eventName = "AssumeRoleWithSAML"
| stats count() as logins by principalId,
      sourceIPAddress, bin(1h)
| sort logins desc
```

알람 조건

- 1시간 내 같은 사용자 5개 이상 IP에서 로그인 → 알람
- 비정상 시간대 (새벽 3시 등) 로그인 → 알람
- 해외 IP에서 로그인 → 알람 (지오 펜싱)
- 실패 5회 연속 → 계정 일시 잠금

CloudTrail

자동 기록

CW Insights

쿼리 도구

Anomaly

이상 패턴

GeoFencing

지역 통제

Part 6 Recap

SSO와 신원 통합 핵심 정리

TAKEAWAY 1

IdP 선택

AWS는 IAM IC, 멀티 클라우드는 Okta, M365는 Azure AD

TAKEAWAY 2

프로토콜

SAML (엔터프라이즈 표준) vs OIDC (현대), 둘 다 사용 가능

TAKEAWAY 3

그룹 매핑

IdP 그룹 → AWS Role 자동 매핑으로 권한 분리 (최소 권한)

TAKEAWAY 4

MFA + JIT + SCIM

다중 인증 강제, 자동 등록, 양방향 동기화로 완성된 신원 관리

NEXT - PART 7

Part 7은 감사와 컴플라이언스입니다. CloudTrail 통합, GDPR, SOC 2, ISO 27001 컴플라이언스 매핑을 10장에 걸쳐 다룹니다.

Part 6

완료

Part 7

Audit (10p)

Part 8-9

Trouble + Labs (15p)

~1h

잔여 시간

CHAPTER 03 - PART 07

Admin Setup – 감사와 컴플라이언스

Enterprise 배포와 거버넌스



Part 7 / 감사와 컴플라이언스

규제 요건 충족

PART OBJECTIVE

CloudTrail 감사 로그, GDPR, SOC 2, ISO 27001 같은 주요 컴플라이언스 표준과 Claude Code의 매핑 관계를 10장에 걸쳐 다룹니다.

TOPIC 1

CloudTrail 통합

AWS Bedrock 호출의 모든 이벤트 로깅과 무결성 보장 방법입니다.

TOPIC 2

주요 컴플라이언스

GDPR, SOC 2, ISO 27001, HIPAA의 핵심 요건과 매핑을 다룹니다.

TOPIC 3

로그 보존

보존 기간, 무결성, 접근 통제, 검색 가능성 요건을 충족합니다.

TOPIC 4

감사 리포트

자동 리포트 생성, 분기 검토, 외부 감사 대응 절차를 다룹니다.

10 Slides

Part 7 분량

~20 min

예상 강의 시간

4 standards

주요 표준

Compliance

핵심 영역

CloudTrail 통합

Bedrock 호출 자동 감사

CloudTrail

```
# 1. CloudTrail이 자동 기록하는 Bedrock 이벤트
- InvokeModel (모델 호출)
- InvokeModelWithResponseStream (스트리밍)
- ListFoundationModels (모델 조회)
```

```
# 2. CloudTrail 설정 (조직 전체 권장)
```

```
aws cloudtrail create-trail \
  --name claude-code-audit-trail \
  --s3-bucket-name audit-logs-bucket \
  --is-multi-region-trail \
  --is-organization-trail \
  --enable-log-file-validation
```

```
# 3. 로그 무결성 보장
```

```
- 파일 단위 SHA-256 해시
- 일일 digest 파일에 모든 해시 기록
- digest는 CloudTrail 키로 서명
- 변조 시 즉시 탐지 가능
```

```
# 4. 기록되는 정보 예시
```

```
{
  "eventTime": "2026-05-25T10:23:45Z",
  "eventName": "InvokeModel",
  "userIdentity": { "principalId": "alice@company.com" },
  "sourceIPAddress": "10.0.1.42",
  "requestParameters": {
    "modelId": "anthropic.claude-sonnet-4-5",
    "inputTokenCount": 1240,
    "outputTokenCount": 380
  }
}
```

Auto

자동 기록

SHA-256

무결성

Org-wide

조직 전체

Per-request

호출별 추적

GDPR 컴플라이언스

유럽 일반 데이터 보호 규정

GDPR 요건	Claude Code 대응	증빙
데이터 처리 위치 (Art 44)	EU 리전 Bedrock 선택 (eu-central-1)	리전 설정 문서
개인정보 학습 미사용	ZDR 옵션 기본, DPA 체결	Anthropic 약관
데이터 삭제 권리 (Art 17)	캐시 미사용, 세션 종료 시 삭제	세션 정책
데이터 처리 기록 (Art 30)	CloudTrail로 모든 처리 기록	감사 로그
DPO 통지 (Art 33)	SIEM 알람 + 사고 대응 절차	인시던트 플레이북

EU region

데이터 위치

ZDR

Zero Data Retention

DPA

데이터 처리 합의

Audit

추적 가능성

SOC 2 매핑

보안 신뢰 서비스 기준

SOC 2 Trust Service	Claude Code 통제	구현
Security (CC6)	IAM, MFA, 네트워크 격리	Part 2, 3, 6
Availability (A1)	AWS Bedrock SLA, 다중 리전	Bedrock 표준
Processing Integrity (PI1)	입력 검증, Hooks, deny 규칙	Part 4
Confidentiality (C1)	TLS, ZDR, secret 분리	Part 2, 3
Privacy (P1)	GDPR 매핑 + DPA	약관 + 정책

SOC 2 TYPE II SOC 2 Type II 감사는 일정 기간(보통 1년) 통제의 운영 효과성을 검증. Claude Code의 통제 항목이 모두 운영되고 있어야 함.

5 TSC
Trust Service

Type II
운영 효과

Annual
연간 감사

Continuous
지속 운영

ISO 27001 매핑

ISMS 핵심 통제

Annex A 통제	Claude Code 적용	문서화
A.5 정책	managed-settings.json 표준화	정책 문서
A.8 자산 관리	라이선스, 사용자, MCP 목록 관리	자산 대장
A.9 접근 통제	IAM, IdP, SCIM, JIT 프로비저닝	신원 다이어그램
A.12 운영 보안	OTel, SIEM, 변경 관리	운영 절차서
A.16 사고 관리	Hooks, 알람, 인시던트 플레이북	플레이북

Annex A
114 통제

ISMS
관리 체계

PDCA
지속 개선

Certifiable
인증 가능

로그 보존 정책

컴플라이언스별 기간 요건

컴플라이언스	최소 보존 기간	대상 로그
GDPR	필요한 동안 (목적 한정)	개인정보 처리 기록
SOC 2	1년 (감사 기간)	모든 보안 이벤트
ISO 27001	3년 권장	감사 로그 전체
HIPAA	6년	의료 정보 접근
SOX	7년	금융 거래 관련
국내 정보통신망법	3-12개월	접속 기록

1-7년
보존 범위

S3 Glacier
저장 옵션

Immutable
변조 방지

Searchable
검색 가능

감사 리포트 자동화

월간 자동 생성

Auto report

```
# Lambda + EventBridge로 매월 1일 자동 생성
```

```
# audit-report-generator.py
import boto3, json
from datetime import datetime, timedelta
```

```
def generate_monthly_report():
    cloudtrail = boto3.client('cloudtrail')
    end = datetime.now()
    start = end - timedelta(days=30)
```

```
    # Bedrock 이벤트 조회
    events = cloudtrail.lookup_events(
        LookupAttributes=[
            {'AttributeKey': 'EventSource', 'AttributeValue':
'bedrock.amazonaws.com'}
        ],
        StartTime=start,
        EndTime=end
    )
```

```
# 통계 집계
```

```
    report = {
        'total_invocations': len(events['Events']),
        'unique_users': len(set(e['Username'] for e in events['Events'])),
        'by_model': {},
        'security_incidents': 0,
        'policy_violations': 0
    }
```

```
# PDF 생성 후 S3에 저장
```

```
s3 = boto3.client('s3')
s3.put_object(
    Bucket='compliance-reports',
    Key=f'monthly/{end.strftime("%Y-%m")}/audit-report.pdf',
    Body=generate_pdf(report)
)
```

```
# 컴플라이언스팀에 이메일
```

Monthly

자동 생성

CloudTrail

데이터 소스

PDF

출력 형식

Email

배포

데이터 분류와 처리

민감도별 차등 대응

분류	예시	Claude Code 정책
Public	공개 문서, OSS 코드	제한 없음
Internal	사내 코드, 일반 문서	표준 정책 적용
Confidential	고객 데이터, 영업 비밀	DLP Hook + 권한 강화
Restricted	개인정보, 금융 정보	Claude Code 사용 금지

GUIDANCE

데이터 분류 시스템을 먼저 구축하고 Claude Code 사용 가이드라인에 매핑.
Restricted 데이터가 들어간 디렉토리는 settings.json deny로 명시적 차단.

4 tiers
분류 단계

Internal+
권장 범위

DLP
민감 차단

Restricted
사용 금지

외부 감사 대응 준비

연 1-2회 감사 대비

STEP 01

감사 일정
확정

STEP 02

문서 패키지
준비

STEP 03

시스템 액세스
부여

STEP 04

인터뷰
대응

STEP 05

발견 사항
조치

DOC 1

정책 문서

managed-settings.json,
권한 정책, SSO 설정 정책 문서

DOC 2

운영 절차

인시던트 대응, 변경 관리,
사용자 등록/삭제 절차서

DOC 3

감사 로그

CloudTrail, SIEM 로그,
/cost 리포트 (감사 기간만)

DOC 4

리스크 평가

AI 사용 리스크 평가, DPIA,
SOC 2 통제 보고서

5 stages

감사 단계

4 docs

필수 문서

Annual

주기

Ready

상시 준비

Part 7 Recap

감사와 컴플라이언스 핵심 정리

TAKEAWAY 1

CloudTrail

조직 전체 Trail + log
validation으로 무결성 보
장된 감사 로그

TAKEAWAY 2

표준 매핑

GDPR, SOC 2, ISO
27001 각 통제 항목을
Claude Code에 매핑

TAKEAWAY 3

데이터 분류

4단계 분류 시스템,
Restricted는 deny로 명시
차단

TAKEAWAY 4

상시 준비

월간 자동 리포트, 문서 상시
유지로 외부 감사 즉시 대응

NEXT - PART 8

Part 8은 트러블슈팅입니다. 자주 발생하는 문제와 대응법을 5장에 걸쳐 다룹니다.

Part 7

완료

Part 8

Troubleshoot (5p)

Part 9

Recap + Labs (10p)

~40 min

잔여 시간

CHAPTER 03 - PART 08

Admin Setup – 트러블슈팅

Enterprise 배포와 거버넌스



Part 8 / 트러블슈팅

자주 발생하는 문제와 대응

증상	원인	대응
인증 실패	API Key 만료 또는 IAM 정책 부족	doctor + IAM 정책 확인
Bedrock access denied	모델 액세스 미신청	Model access 페이지 확인
TLS verification failed	사내 CA 미설치	NODE_EXTRA_CA_CERTS
Proxy connection refused	프록시 URL/포트 오류	curl로 직접 테스트
Permission denied (Bash)	managed-settings deny	정책 검토

doctor

1차 진단

5 common

대표 문제

Solvable

해결 가능

Docs

문서 참고

인증 트러블슈팅

자격증명 문제 단계별 진단

Auth debug

Symptom: "Authentication failed" 또는 "No credentials"

1. doctor로 시작

\$ `claude doctor`

Auth:

Provider: ? → 이 값이 핵심 단서

2. Bedrock 사용 시

\$ `aws sts get-caller-identity` # 자격증명 확인

\$ `aws bedrock list-foundation-models --region ap-northeast-2`

둘 다 성공해야 정상

3. SSO 사용 시

\$ `aws sso login --profile claude-code`

브라우저 열림 → 로그인 → 토큰 발급

\$ `ls -la ~/.aws/sso/cache/` # 토큰 파일 존재 확인

4. API Key 사용 시 (Pro/Max)

\$ `ls -la ~/.claude/oauth.json` # 토큰 파일

\$ `claude /logout && claude /login` # 재로그인

5. 환경변수 충돌 확인

\$ `env | grep -i 'claude\|anthropic\|aws_'`

의도치 않은 환경변수가 우선 적용될 수 있음

6. 디버그 모드

\$ `claude --debug`

Task 호출 내부와 자격증명 흐름 출력

doctor

시작점

sts caller

AWS 확인

sso login

SSO 재로그인

--debug

상세 로그

네트워크 트러블슈팅

연결 문제 진단 흐름

STEP 01

doctor로
증상 확인

STEP 02

curl로 도메인
직접 테스트

STEP 03

프록시 확인
+ 인증서

STEP 04

DNS 해석
확인

STEP 05

방화벽
로그 점검

```
# 종합 진단 스크립트
#!/bin/bash
echo "=== Network diagnosis ==="
```

```
# 1. doctor
claude doctor 2>&1 | grep -A 5 'Network'
```

```
# 2. 도메인별 연결 테스트
for d in api.anthropic.com bedrock-runtime.ap-northeast-2.amazonaws.com; do
  echo "Testing $d ..."
  curl -sS -o /dev/null -w " Status: %{http_code}, Time: %{time_total}s\n" \
    --connect-timeout 5 \
    "https://$d/" || echo " FAIL"
done
```

```
# 3. 인증서 검증
openssl s_client -connect api.anthropic.com:443 \
  -CAfile ${NODE_EXTRA_CA_CERTS} </dev/null 2>&1 | \
  grep 'Verify return code'
```

```
# 4. DNS
dig +short api.anthropic.com
dig +short bedrock-runtime.ap-northeast-2.amazonaws.com
```

```
# 5. 프록시 환경변수
env | grep -i proxy
```

[net-debug.sh](#)

Script

자동 진단

curl

연결 확인

openssl

인증서

dig

DNS

성능 문제 진단

응답 지연과 토큰 폭증

ISSUE 1

응답 지연

프록시 경유 + 인증서 검증
+ 모델 선택. p99 지연 측정
으로 원인 분리.

ISSUE 2

토큰 폭증

컨텍스트 누적, 큰 파일
Read, Subagent 무한 루
프. /cost로 추적.

ISSUE 3

rate limit

API 호출 한도 초과. 모델별
한도 확인 또는 backoff 적
용.

ISSUE 4

메모리 부족

큰 코드베이스 분석 시 발생.
Subagent 위임 또는
chunked 처리.

METRICS

OpenTelemetry로 모든 메트릭을 수집해 두면 성능 문제 원인 파악이 쉬워짐.

p50/p99 응답 시간, 토큰 사용량, 도구 호출 횟수를 항상 모니터링.

Latency

응답

Tokens

사용량

Rate

호출 한도

Memory

메모리

에스컬레이션 경로

문제가 풀리지 않을 때

STEP 01

doctor +
자가 진단

STEP 02

사내 IT
Help Desk

STEP 03

본 워크샵
FAQ + 자료

STEP 04

Anthropic
Developer Forum

STEP 05

AWS Support
(Bedrock 한정)

단계	연락처/위치	대상 문제
Self-help	claude doctor, docs.claude.com	기본 모든 문제
Internal IT	helpdesk@company.com	사내망/프록시/인증서
Anthropic	support.anthropic.com	Claude Code 자체
AWS Support	AWS Console → Support	Bedrock 모델/IAM

Self → Org
단계 진행

Self-help
먼저 시도

Specific
대상별 채널

Documented
재현 정보

CHAPTER 03

Admin Setup – Summary / LAB

Enterprise 배포와 거버넌스



Chapter 3 핵심 요약

9개 Part 110장의 압축본

DEPLOY

배포와 자격증명

4가지 방식 선택, 사내 미러,
Bedrock 통합, SSO 적용

SECURE

네트워크와 거버넌스

프록시, 인증서, managed-
settings 강제, Hooks로
DLP

OBSERVE

모니터링과 비용

OTel + SIEM, /cost 추적,
예산 알람, 자동 차단

COMPLIANCE

신원과 감사

IdP 통합, MFA, SCIM,
CloudTrail, SOC 2/ISO 매
핑

OUTCOME

여러분은 이제 Enterprise 환경에 Claude Code를 안전하고 통제 가능한 형태로 배포할 수 있습니다.
9개 Part의 모든 패턴을 본인 조직에 적용 가능합니다.

9 Parts

완료

110 Slides

Chapter 3

~4h

강의 시간

3 Labs

실습 시작

Hands-on Labs / 3종

직접 손에 익히는 시간

Lab	내용	예상 시간
Lab 1 / 사내 미러 구축	Verdaccio로 npm 미러 + Claude Code 설치	30분
Lab 2 / Bedrock SSO 통합	IAM Identity Center + Bedrock 권한 정책	30분
Lab 3 / managed-settings 거버넌스	권한 정책 + DLP Hook + OTel 수집	30분

TOTAL

3개 랩 합산 약 90분. 본인 조직 환경에 맞춰 진행하면 실무 적용도가 높아집니다.

3 Labs

실습 수

~90 min

총 시간

Hands-on

실습 중심

Mentor

AWS에 지원 요청 가능

Lab 1 / 사내 미러 구축

Verdaccio + Claude Code 배포 (30분)

STEP 01

Verdaccio
설치

STEP 02

@anthropic-ai
프록시 설정

STEP 03

npm registry
변경

STEP 04

Claude Code
설치

STEP 05

동작 검증

Lab 1 steps

```
# 1. Verdaccio 설치 (Lab VM에서)
$ npm install -g verdaccio
$ verdaccio --listen 4873 &
```

```
# 2. config 수정 (~/.config/verdaccio/config.yaml)
packages:
  '@anthropic-ai/*':
    access: $all
    publish: $authenticated
    proxy: npmjs
  '**':
    proxy: npmjs
```

```
# 3. 클라이언트 npm registry 설정
$ npm config set registry http://localhost:4873
```

```
# 4. Claude Code 설치 (미러 경유)
$ npm install -g @anthropic-ai/claude-code
```

```
# 5. 검증
$ claude --version
$ ls -la \
  ~/.config/verdaccio/storage/@anthropic-ai/claude-code/
# 캐싱 확인
```

30 min

예상 소요

Verdaccio

미러 도구

Cache

오프라인 사용

Verify

동작 검증

Lab 2 / Bedrock SSO 통합

IAM Identity Center 연동 (30분)

Lab 2 steps

1. IAM Identity Center 활성화 (AWS Console)

Services → IAM Identity Center → Enable

2. Permission Set 생성

Name: ClaudeCodeWorkshop

AWS managed policy 또는 인라인 정책:

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": ["bedrock:InvokeModel", "bedrock:ListFoundationModels"],
    "Resource": [
      "arn:aws:bedrock:::foundation-model/anthropic.claude-sonnet-4-5*",
      "arn:aws:bedrock:::foundation-model/anthropic.claude-haiku-4-5*"
    ]
  }]
}
```

3. 사용자 생성 및 Permission Set 할당

Workshop 그룹 → ClaudeCodeWorkshop 부여

4. 사용자 PC에서 SSO 로그인

\$ aws sso configure # 또는 ~/.aws/config 직접 편집

\$ aws sso login --profile claude-workshop

5. Claude Code 실행

\$ export CLAUDE_CODE_USE_BEDROCK=1

\$ export AWS_PROFILE=claude-workshop

\$ export AWS_REGION=us-east-1

\$ claude doctor # Provider: AWS Bedrock 확인

\$ claude # 첫 세션 시작

30 min

예상 소요

IAM IC

SSO 도구

Bedrock

대상 서비스

Verified

doctor 확인

Lab 3 / managed-settings 거버넌스

권한 + DLP + OTel (30분)

Lab 3 steps

```
# 1. managed-settings.json 작성 (Lab VM)
sudo tee /etc/claude/managed-settings.json <<EOF
{
  "permissions": {
    "allow": ["Read(**)", "Grep", "Glob", "Bash(npm test:*)"],
    "ask": ["Edit(**)", "Write(**)"],
    "deny": ["Bash(rm -rf:*)", "Read(**/.env*)"]
  },
  "hooks": {
    "PreToolUse": [{
      "matcher": "Bash|Edit|Write",
      "hooks": [{ "type": "command", "command": "/opt/claude/dlp.sh" }]
    }]
  },
  "telemetry": {
    "enabled": true,
    "endpoint": "https://otel.lab.local:4318"
  }
}
EOF
```

```
# 2. DLP Hook 스크립트 작성
sudo tee /opt/claude/dlp.sh <<'EOF'
#!/bin/bash
INPUT=$(cat)
if echo "$INPUT" | grep -qE 'AKIA[0-9A-Z]{16}'; then
  echo '{"action":"block","reason":"AWS key"}' >&2
  exit 1
fi
exit 0
EOF
sudo chmod +x /opt/claude/dlp.sh
```

```
# 3. 검증 시나리오
> 다음 코드를 src/aws.js에 작성해줘:
AKIAIOSFODNN7EXAMPLE
# 결과: DLP Hook이 차단 (정상 동작)
```

30 min

예상 소요

3 controls

권한+Hook+OTel

DLP test

차단 검증

Production

실무 패턴

자주 묻는 질문

Chapter 3 관련 FAQ

FAQ

Q1. 사내망에서 외부 인터넷 차단인데 Claude Code 사용 가능한가요?

A. 네, Bedrock VPC Endpoint를 통해 AWS 사설망 내에서만 통신 가능합니다.
npm 미러는 사내에 구축, MCP 서버도 사내로.

Q2. Claude Code 도입 비용 예산을 어떻게 잡아야 하나요?

A. 1인당 월 \$27 기준 + 30% 안전 마진. Opus 사용은 별도 통제.
PoC 단계에서 실측 후 본 도입 시 조정 권장.

Q3. SSO와 OAuth Pro 중 선택은?

A. 10인 이상은 Bedrock + IAM Identity Center 권장.
1-5인 PoC라면 OAuth Pro로 간단히 시작 가능.

Q4. 개발자가 managed-settings를 우회할 수 있나요?

A. /etc/claude 디렉토리는 root 권한 필요. 일반 사용자는 불가능.
deny 규칙은 항상 승리하므로 우회 방법 없음.

Q5. 컴플라이언스 감사 자료를 어디서 얻나요?

A. CloudTrail (필수), OpenTelemetry (선택), managed-settings 문서.
Anthropic에서 SOC 2 보고서 요청 가능.

5+ Q

대표 질문

Docs

공식 문서

Support

지원 채널

Practice

실무 적용

실행 로드맵

도입 단계별 4분기 계획

분기	목표	주요 활동
Q1 / PoC	5-10인 시범 도입	API Key + 사용 패턴 관찰
Q2 / Foundation	Cloud 자격증명 전환	Bedrock + SSO 통합
Q3 / Governance	권한 정책과 감사 시스템	managed-settings + SIEM
Q4 / Scale	전 조직 확산	교육 + 자동화 + 모니터

PHILOSOPHY

한 번에 모든 것을 하지 마시고 점진적으로 도입하세요. 각 분기마다 학습한 내용을 다음 분기 계획에 반영하는 사이클이 가장 효과적입니다.

Q1-Q4

단계별

PoC first

시범 우선

Iterate

반복 개선

Sustainable

지속 가능

References

참고 자료 및 출처

REFERENCES

Official Documentation

<https://code.claude.com/docs/ko/admin-setup>
<https://code.claude.com/docs/ko/deployment>
<https://code.claude.com/docs/ko/iam-integration>
<https://code.claude.com/docs/ko/managed-settings>

AWS Resources

Bedrock User Guide: <https://docs.aws.amazon.com/bedrock/>
IAM Identity Center: <https://docs.aws.amazon.com/singlesignon/>
CloudTrail: <https://docs.aws.amazon.com/cloudtrail/>

Compliance References

SOC 2 Trust Services Criteria
ISO 27001 Annex A controls
GDPR official text: gdpr-info.eu

Internal Tooling Examples

Verdaccio: <https://verdaccio.org/>
HashiCorp Vault: <https://www.vaultproject.io/>

Docs

공식 문서

AWS

클라우드

Compliance

표준

Tools

오픈소스

Thank you!

Choi, WooHyung / Prin. Solutions Architect / AWS Korea

whchoi@amazon.com

<https://linkedin.com/in/woohyungchoi>

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved. Amazon Confidential and Trademark.

